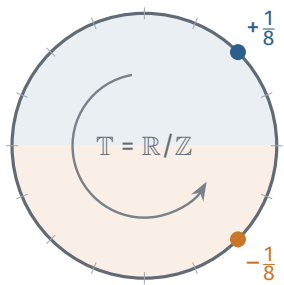
# TFHE from Scratch

A C++ implementation of gate bootstrapping over the torus

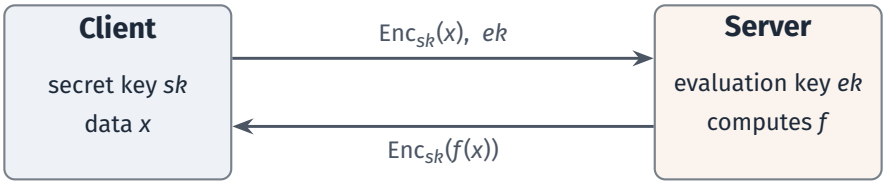
Matheus de Oliveira Saldanha

October 29, 2026

MO422 – Cryptographic Algorithms
Instituto de Computação – Unicamp

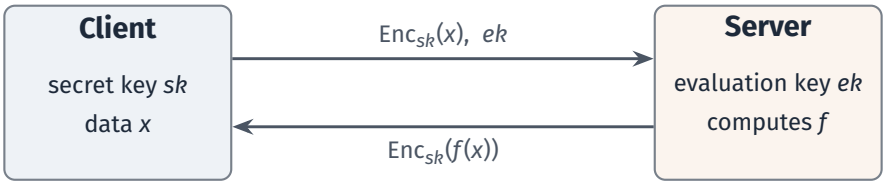


# Computing on data you cannot see




the server sees neither $x$ nor $f(x)$

# Computing on data you cannot see

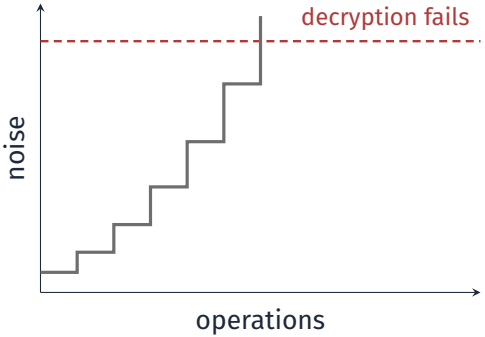



the server sees neither $x$ nor $f(x)$

**Fully homomorphic encryption (FHE):** any $f$, not just sums or products. Asked by Rivest, Adleman and Dertouzos (1978); first solved by Gentry (2009); all practical schemes today are based on lattices (LWE).

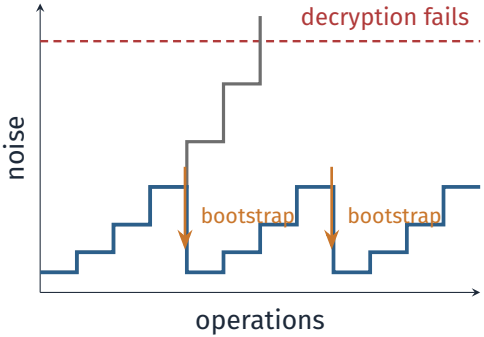
# Every lattice ciphertext carries noise

- The noise is the security: without it, LWE is linear algebra.
- Homomorphic operations make it grow; past a threshold, decryption fails.



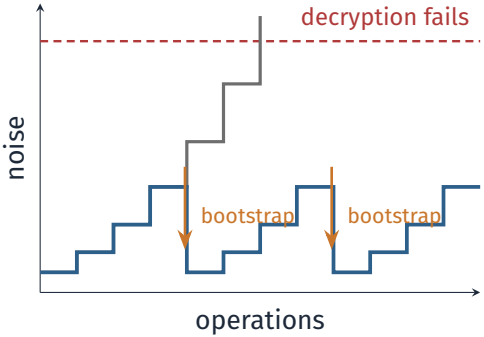
# Every lattice ciphertext carries noise

- The noise is the security: without it, LWE is linear algebra.
- Homomorphic operations make it grow; past a threshold, decryption fails.
- **Bootstrapping** (Gentry, 2009): evaluate the decryption homomorphically to get a *fresh* ciphertext of the same message.



# Every lattice ciphertext carries noise

- The noise is the security: without it, LWE is linear algebra.
- Homomorphic operations make it grow; past a threshold, decryption fails.
- **Bootstrapping** (Gentry, 2009): evaluate the decryption homomorphically to get a *fresh* ciphertext of the same message.
- **TFHE** (Chillotti et al., 2016/2020): bootstrap after every Boolean gate, in tens of milliseconds, so circuits of any depth work.



# This project

**Goal** (from the proposal)

- Study TFHE and implement it from scratch in C++.
- Use the same parameters as the original library (128-bit set: $n = 630$, $N = 1024$).
- Compare with the original on four criteria.

**Result**

- Header-only C++17, no dependencies, 526 lines of code.
- 10 unit tests, one per layer; an encrypted $n$-bit adder

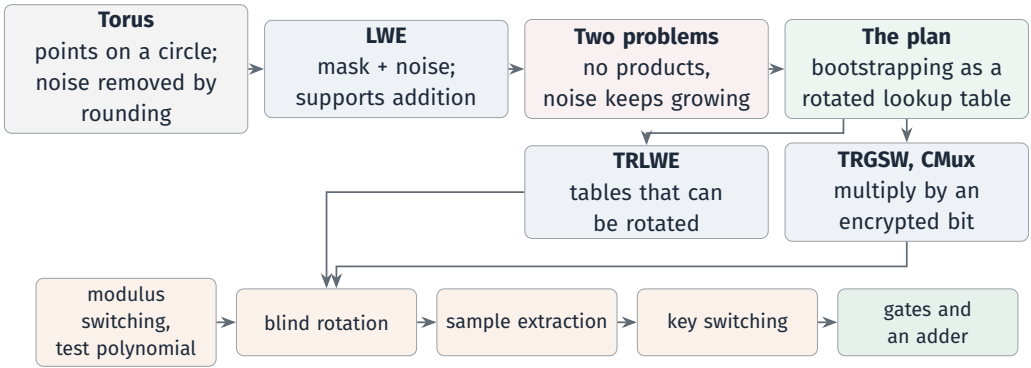
**Correctness**
error rate

**Performance**
time per gate

**Memory**
key sizes

**Noise**
measured vs. theory

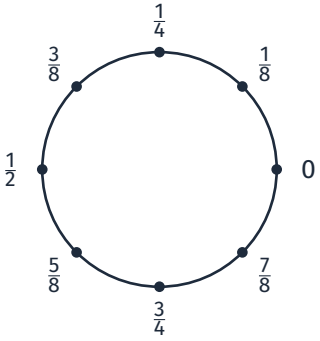
# Roadmap: each concept solves a problem raised by the previous one




then: **implementation** (FFT, offset trick, frequency-domain keys) $\rightarrow$ **evaluation** (correctness, time, memory, noise)

# The scheme

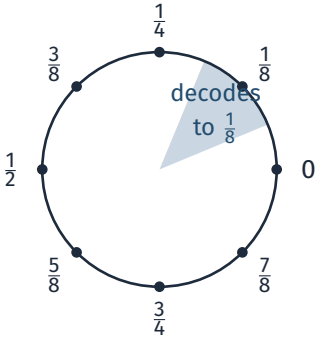
# The torus $\mathbb{T} = \mathbb{R}/\mathbb{Z}$




**Why a circle?** Messages can be added, and small errors are removed by rounding.
$(0.25 \equiv 1.25 \equiv -0.75)$

- A $\mathbb{Z}$-module: $t + t'$ and $k \cdot t$ exist, but not $t \cdot t'$ ($\frac{1}{2} \equiv \frac{3}{2}$, yet $\frac{1}{2} \cdot \frac{1}{2} \not\equiv \frac{3}{2} \cdot \frac{1}{2}$).

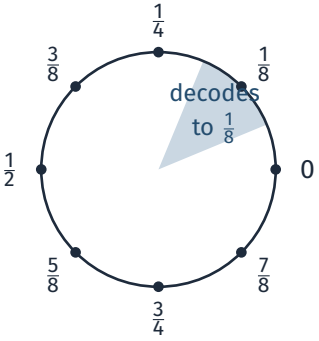
# The torus $\mathbb{T} = \mathbb{R}/\mathbb{Z}$




**Why a circle?** Messages can be added, and small errors are removed by rounding.
$(0.25 \equiv 1.25 \equiv -0.75)$

- A $\mathbb{Z}$-module: $t + t'$ and $k \cdot t$ exist, but not $t \cdot t'$ ($\frac{1}{2} \equiv \frac{3}{2}$, yet $\frac{1}{2} \cdot \frac{1}{2} \not\equiv \frac{3}{2} \cdot \frac{1}{2}$).
- Message $m \in \mathbb{Z}_M \mapsto m/M$; decoding rounds and tolerates errors $< \frac{1}{2M}$.

# The torus $\mathbb{T} = \mathbb{R}/\mathbb{Z}$




**Why a circle?** Messages can be added, and small errors are removed by rounding.
$(0.25 \equiv 1.25 \equiv -0.75)$

- A $\mathbb{Z}$-module: $t + t'$ and $k \cdot t$ exist, but not $t \cdot t'$ ($\frac{1}{2} \equiv \frac{3}{2}$, yet $\frac{1}{2} \cdot \frac{1}{2} \not\equiv \frac{3}{2} \cdot \frac{1}{2}$).
- Message $m \in \mathbb{Z}_M \mapsto m/M$; decoding rounds and tolerates errors $< \frac{1}{2M}$.

**In code: `uint32_t`; overflow = mod 1**

$0.75 + 0.5 \equiv 0.25$:

`0xC0000000 + 0x80000000` $\longrightarrow$ `0x40000000`

# LWE: hiding a message with a mask and noise

**First attempt:** secret bits $s \in \{0,1\}^n$,
public random $\mathbf{a}$,

$$b = \langle \mathbf{a}, s \rangle + \mu$$

only the key owner can remove the mask $\langle \mathbf{a}, s \rangle$...

# LWE: hiding a message with a mask and noise

**First attempt:** secret bits $s \in \{0,1\}^n$, public random $\mathbf{a}$,

$$b = \langle \mathbf{a}, s \rangle + \mu$$

only the key owner can remove the mask $\langle \mathbf{a}, s \rangle$…

Broken: every ciphertext is a *linear equation* in $s$; $n$ of them $\Rightarrow$ Gaussian elimination.

# LWE: hiding a message with a mask and noise

**First attempt:** secret bits $s \in \{0,1\}^n$, public random $\mathbf{a}$,

$$b = \langle \mathbf{a}, s \rangle + \mu$$

only the key owner can remove the mask $\langle \mathbf{a}, s \rangle$…

Broken: every ciphertext is a *linear equation* in $s$; $n$ of them $\Rightarrow$ Gaussian elimination.

**Fix** (Regev 2005): make every equation slightly wrong,

$$b = \langle \mathbf{a}, s \rangle + \mu + e, \qquad e \sim \mathcal{N}(0, \sigma^2)$$

# LWE: hiding a message with a mask and noise

**First attempt:** secret bits $s \in \{0,1\}^n$, public random $\mathbf{a}$,

$$b = \langle \mathbf{a}, s \rangle + \mu$$

only the key owner can remove the mask $\langle \mathbf{a}, s \rangle$…

Broken: every ciphertext is a *linear equation* in $s$; $n$ of them $\Rightarrow$ Gaussian elimination.

**Fix** (Regev 2005): make every equation slightly wrong,

$$b = \langle \mathbf{a}, s \rangle + \mu + e, \qquad e \sim \mathcal{N}(0, \sigma^2)$$

**Decryption = phase, then round**

$\varphi_s(c) = b - \langle \mathbf{a}, s \rangle = \mu + e$

the noise is the security; rounding removes it

# LWE: hiding a message with a mask and noise

**First attempt:** secret bits $s \in \{0,1\}^n$, public random $\mathbf{a}$,

$$b = \langle \mathbf{a}, s \rangle + \mu$$

only the key owner can remove the mask $\langle \mathbf{a}, s \rangle$…

Broken: every ciphertext is a *linear equation* in $s$; $n$ of them $\Rightarrow$ Gaussian elimination.

**Fix** (Regev 2005): make every equation slightly wrong,

$$b = \langle \mathbf{a}, s \rangle + \mu + e, \qquad e \sim \mathcal{N}(0, \sigma^2)$$

**Decryption = phase, then round**

$\varphi_s(c) = b - \langle \mathbf{a}, s \rangle = \mu + e$

the noise is the security; rounding removes it

**Toy example: 8-bit torus, $n = 4$, $\mathbb{Z}_4$**

$s = (1, 0, 1, 1)$, $\mathbf{a} = (100, 37, 201, 54)$,

$\mu = 64$, $e = 3$

$b = 355 + 64 + 3 \equiv 166$

phase $= 166 - 355 \equiv \mathbf{67} \approx 64 \Rightarrow$ message 1

# What LWE can and cannot do

**Can: linear operations, for free**

- The phase is linear:
  $\varphi(c_1 + c_2) = \varphi(c_1) + \varphi(c_2)$.
- $\frac{3}{8} + \frac{6}{8} \equiv \frac{1}{8}$: addition mod 8 without the key.
- Constants via the trivial sample $(\mathbf{0}, \mu)$.

# What LWE can and cannot do

**Can: linear operations, for free**

- The phase is linear:
  $\varphi(c_1 + c_2) = \varphi(c_1) + \varphi(c_2)$.
- $\frac{3}{8} + \frac{6}{8} \equiv \frac{1}{8}$: addition mod 8 without the key.
- Constants via the trivial sample $(\mathbf{0}, \mu)$.

**Cannot:**

1. no products: components are torus elements;
2. noise grows: variances add at every operation.

# What LWE can and cannot do

**Can: linear operations, for free**

- The phase is linear: $\varphi(c_1 + c_2) = \varphi(c_1) + \varphi(c_2)$.
- $\frac{3}{8} + \frac{6}{8} \equiv \frac{1}{8}$: addition mod 8 without the key.
- Constants via the trivial sample $(\mathbf{0}, \mu)$.

**Cannot:**

1. no products: components are torus elements;
2. noise grows: variances add at every operation.

**TFHE's bootstrapping fixes both**

In: LWE of a noisy phase $\varphi$.

Out: LWE of $f(\varphi)$ with fresh noise.

For gates, $f$ = *which half of the circle?*

- nonlinear $\Rightarrow$ fixes 1
- fresh noise $\Rightarrow$ fixes 2

# What LWE can and cannot do

**Can: linear operations, for free**

- The phase is linear: $\varphi(c_1 + c_2) = \varphi(c_1) + \varphi(c_2)$.
- $\frac{3}{8} + \frac{6}{8} \equiv \frac{1}{8}$: addition mod 8 without the key.
- Constants via the trivial sample $(\mathbf{0}, \mu)$.

**Cannot:**

1. no products: components are torus elements;
2. noise grows: variances add at every operation.

**TFHE's bootstrapping fixes both**

In: LWE of a noisy phase $\varphi$.

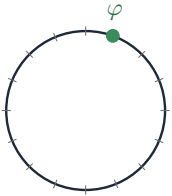
Out: LWE of $f(\varphi)$ with fresh noise.

For gates, $f$ = *which half of the circle?*

- nonlinear $\Rightarrow$ fixes 1
- fresh noise $\Rightarrow$ fixes 2

But how can a server compute $f(\varphi)$ without knowing $\varphi$?

# The plan: bootstrapping as an encrypted table lookup

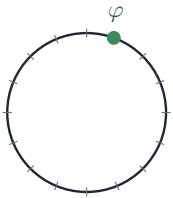



encrypted phase
(LWE sample)

| $v_0$ | $v_1$ | $v_2$ | $v_3$ | $v_4$ | $v_5$ | $v_6$ | $v_7$ |
|---|---|---|---|---|---|---|---|

test polynomial $v$:
one answer per position

# The plan: bootstrapping as an encrypted table lookup




encrypted phase
(LWE sample)

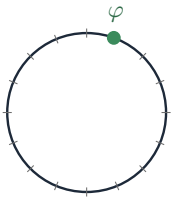
① round

$\bar{\varphi} = 3$

| $v_0$ | $v_1$ | $v_2$ | $v_3$ | $v_4$ | $v_5$ | $v_6$ | $v_7$ |
|---|---|---|---|---|---|---|---|

test polynomial $v$:
one answer per position

# The plan: bootstrapping as an encrypted table lookup




① round

$\bar{\varphi} = 3$

| $v_0$ | $v_1$ | $v_2$ | $v_3$ | $v_4$ | $v_5$ | $v_6$ | $v_7$ |
|---|---|---|---|---|---|---|---|

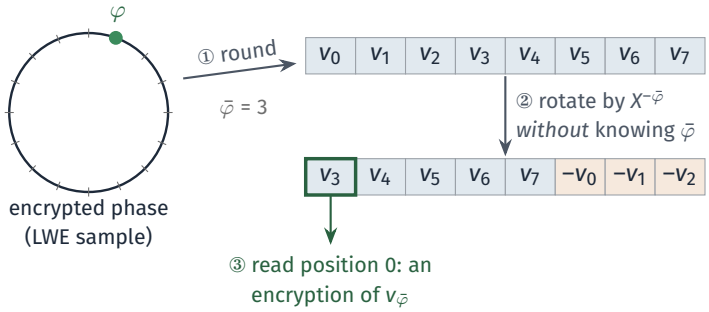
② rotate by $X^{-\bar{\varphi}}$
*without* knowing $\bar{\varphi}$

| $v_3$ | $v_4$ | $v_5$ | $v_6$ | $v_7$ | $-v_0$ | $-v_1$ | $-v_2$ |
|---|---|---|---|---|---|---|---|

test polynomial $v$:
one answer per position

$X^{-\bar{\varphi}} \cdot v$: the answer
for $\bar{\varphi} = 3$ is now first

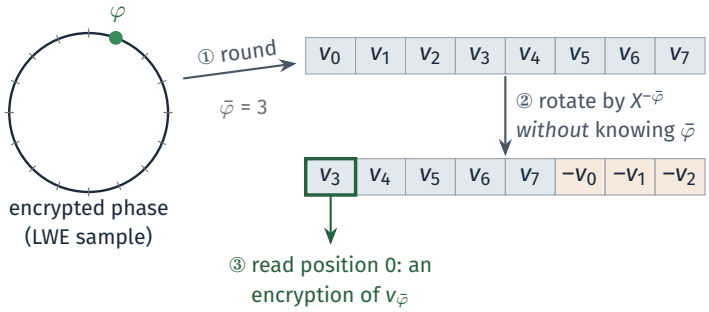
# The plan: bootstrapping as an encrypted table lookup




test polynomial $v$:
one answer per position

$X^{-\bar{\varphi}} \cdot v$: the answer
for $\bar{\varphi} = 3$ is now first

# The plan: bootstrapping as an encrypted table lookup

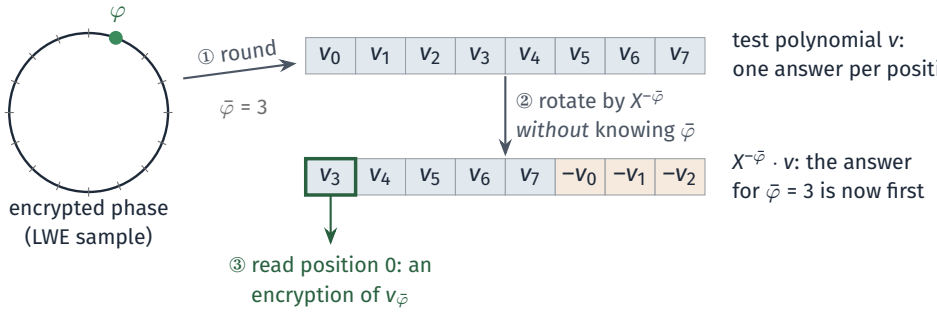



test polynomial $v$:
one answer per position

$X^{-\bar{\varphi}} \cdot v$: the answer
for $\bar{\varphi} = 3$ is now first

**Tool (a):** an encrypted table that can be rotated cheaply and without growing noise
$\rightarrow$ **TRLWE**

# The plan: bootstrapping as an encrypted table lookup



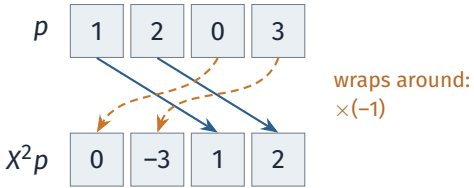

**Tool (a):** an encrypted table that can be rotated cheaply and without growing noise $\rightarrow$ **TRLWE**

**Tool (b):** rotate by a secret amount, i.e., "if encrypted bit then rotate" $\rightarrow$ **TRGSW, CMux**

# Tool (a): tables that can be rotated (polynomials and TRLWE)

Table = polynomial $v_0 + v_1 X + \ldots$;
rotation = multiplication by $X$.

**Negacyclic rotation** ($X^N = -1$, exponents mod $2N$)

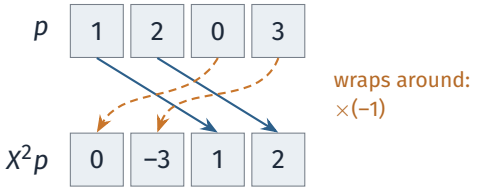



$$(1 + 2X + 3X^3) \cdot X^2 = -3X + X^2 + 2X^3$$

# Tool (a): tables that can be rotated (polynomials and TRLWE)

Table = polynomial $v_0 + v_1 X + \dots$;
rotation = multiplication by $X$.

**Negacyclic rotation** ($X^N = -1$, exponents mod $2N$)




$$(1 + 2X + 3X^3) \cdot X^2 = -3X + X^2 + 2X^3$$

**TRLWE under** $s' \in \mathbb{B}_N[X]$

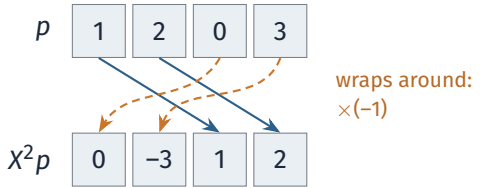
$c = (a, b)$, $b = a \cdot s' + \mu + e$ in $\mathbb{T}_N[X]$

phase $b - a \cdot s' = \mu + e$: a whole polynomial

# Tool (a): tables that can be rotated (polynomials and TRLWE)

Table = polynomial $v_0 + v_1 X + \dots$;
rotation = multiplication by $X$.

**Negacyclic rotation** ($X^N = -1$, exponents mod $2N$)




$(1 + 2X + 3X^3) \cdot X^2 = -3X + X^2 + 2X^3$

**TRLWE under $s' \in \mathbb{B}_N[X]$**

$c = (a, b)$, $b = a \cdot s' + \mu + e$ in $\mathbb{T}_N[X]$

phase $b - a \cdot s' = \mu + e$: a whole polynomial

- Rotation for free: $\varphi(X^e a, X^e b) = X^e \, \varphi(c)$.
- Noise is only permuted, never amplified.
- One sample holds a table of $N$ values.

# Tool (b): rotating by a secret amount

The rotation depends on $\varphi = b - \sum_i a_i s_i$, i.e., on the secret bits $s_i$. It can be done one bit at a time:

$$\mathrm{ACC} \leftarrow \begin{cases} X^{k_i} \cdot \mathrm{ACC} & \text{if } s_i = 1 \\ \mathrm{ACC} & \text{if } s_i = 0 \end{cases} = \mathrm{ACC} + s_i \cdot (X^{k_i} - 1) \cdot \mathrm{ACC}$$

# Tool (b): rotating by a secret amount

The rotation depends on $\varphi = b - \sum_i a_i s_i$, i.e., on the secret bits $s_i$. It can be done one bit at a time:

$$\text{ACC} \leftarrow \begin{cases} X^{k_i} \cdot \text{ACC} & \text{if } s_i = 1 \\ \text{ACC} & \text{if } s_i = 0 \end{cases} = \text{ACC} + s_i \cdot (X^{k_i} - 1) \cdot \text{ACC}$$

$\Rightarrow$ we must **multiply a TRLWE ciphertext by an encrypted bit**.

# Tool (b): rotating by a secret amount

The rotation depends on $\varphi = b - \sum_i a_i s_i$, i.e., on the secret bits $s_i$. It can be done one bit at a time:

$$\text{ACC} \leftarrow \begin{cases} X^{k_i} \cdot \text{ACC} & \text{if } s_i = 1 \\ \text{ACC} & \text{if } s_i = 0 \end{cases} = \text{ACC} + s_i \cdot (X^{k_i} - 1) \cdot \text{ACC}$$

$\Rightarrow$ we must **multiply a TRLWE ciphertext by an encrypted bit**.

**Obstacle 1**

The bit cannot be an LWE/TRLWE ciphertext: torus $\times$ torus does not exist.

# Tool (b): rotating by a secret amount

The rotation depends on $\varphi = b - \sum_i a_i s_i$, i.e., on the secret bits $s_i$. It can be done one bit at a time:

$$\text{ACC} \leftarrow \begin{cases} X^{k_i} \cdot \text{ACC} & \text{if } s_i = 1 \\ \text{ACC} & \text{if } s_i = 0 \end{cases} = \text{ACC} + s_i \cdot (X^{k_i} - 1) \cdot \text{ACC}$$

$\Rightarrow$ we must **multiply a TRLWE ciphertext by an encrypted bit**.

**Obstacle 1**

The bit cannot be an LWE/TRLWE ciphertext: torus $\times$ torus does not exist.

**Obstacle 2**

Even $k \cdot c$ is dangerous: noise $\times |k|$, and coefficients are 32-bit numbers.

# Tool (b): rotating by a secret amount

The rotation depends on $\varphi = b - \sum_i a_i s_i$, i.e., on the secret bits $s_i$. It can be done one bit at a time:

$$\text{ACC} \leftarrow \begin{cases} X^{k_i} \cdot \text{ACC} & \text{if } s_i = 1 \\ \text{ACC} & \text{if } s_i = 0 \end{cases} = \text{ACC} + s_i \cdot (X^{k_i} - 1) \cdot \text{ACC}$$

$\Rightarrow$ we must **multiply a TRLWE ciphertext by an encrypted bit**.

**Obstacle 1**

The bit cannot be an LWE/TRLWE ciphertext: torus $\times$ torus does not exist.

**Obstacle 2**

Even $k \cdot c$ is dangerous: noise $\times |k|$, and coefficients are 32-bit numbers.

TFHE's answer in three steps: **small digits** $\rightarrow$ **precomputed encryptions (TRGSW)** $\rightarrow$ **external product**

# Step 1: small digits instead of a big factor (gadget decomposition)

**Removes obstacle 2:** only small integers will ever multiply a ciphertext.

**Idea:** write the big factor with small digits

$$t \approx \frac{d_1}{B_g} + \frac{d_2}{B_g^2} + \cdots + \frac{d_\ell}{B_g^\ell}, \qquad d_i \in \left[-\frac{B_g}{2}, \frac{B_g}{2}\right)$$

with error at most $\varepsilon = 1/(2B_g^\ell)$.

Here: $B_g = 2^7$, $\ell = 3$, digits in $[-64, 64)$, $\varepsilon = 2^{-22}$.

**The same idea in base 10,** $\ell = 2$, digits in $[-5, 5)$

$t = 0.7316$

| 7 | 3 | 16 |
|---|---|---|

# Step 1: small digits instead of a big factor (gadget decomposition)

**Removes obstacle 2:** only small integers will ever multiply a ciphertext.

**Idea:** write the big factor with small digits

$$t \approx \frac{d_1}{B_g} + \frac{d_2}{B_g^2} + \cdots + \frac{d_\ell}{B_g^\ell}, \qquad d_i \in \left[-\frac{B_g}{2}, \frac{B_g}{2}\right)$$

with error at most $\varepsilon = 1/(2B_g^\ell)$.

Here: $B_g = 2^7$, $\ell = 3$, digits in $[-64, 64)$, $\varepsilon = 2^{-22}$.

**The same idea in base 10,** $\ell = 2$, digits in $[-5, 5)$

$t = 0.7316$

| 7 | 3 | 16 |
|---|---|---|

$7 \geq 5$: out of range

# Step 1: small digits instead of a big factor (gadget decomposition)

**Removes obstacle 2:** only small integers will ever multiply a ciphertext.

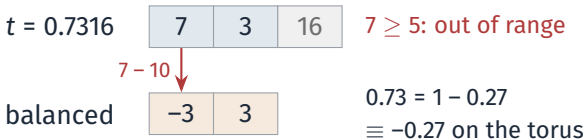
**Idea:** write the big factor with small digits

$$t \approx \frac{d_1}{B_g} + \frac{d_2}{B_g^2} + \cdots + \frac{d_\ell}{B_g^\ell}, \qquad d_i \in \left[-\frac{B_g}{2}, \frac{B_g}{2}\right)$$

with error at most $\varepsilon = 1/(2B_g^\ell)$.

Here: $B_g = 2^7$, $\ell = 3$, digits in $[-64, 64)$, $\varepsilon = 2^{-22}$.

**The same idea in base 10,** $\ell = 2$, digits in $[-5, 5)$



# Step 1: small digits instead of a big factor (gadget decomposition)

**Removes obstacle 2:** only small integers will ever multiply a ciphertext.

**Idea:** write the big factor with small digits

$$t \approx \frac{d_1}{B_g} + \frac{d_2}{B_g^2} + \ldots + \frac{d_\ell}{B_g^\ell}, \qquad d_i \in \left[-\frac{B_g}{2}, \frac{B_g}{2}\right)$$

with error at most $\varepsilon = 1/(2B_g^\ell)$.

Here: $B_g = 2^7$, $\ell = 3$, digits in $[-64, 64)$, $\varepsilon = 2^{-22}$.

**The same idea in base 10,** $\ell = 2$, digits in $[-5, 5)$

$t = 0.7316$ | 7 | 3 | 16 | $7 \geq 5$: out of range

$7 - 10$

balanced | −3 | 3 |

$0.73 = 1 - 0.27$
$\equiv -0.27$ on the torus

check: $-\frac{3}{10} + \frac{3}{100} = -0.27 \equiv 0.73$,
error $0.0016 \leq \varepsilon = 0.005$

# Step 1: small digits instead of a big factor (gadget decomposition)

**Removes obstacle 2:** only small integers will ever multiply a ciphertext.

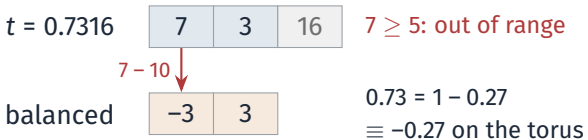
**Idea:** write the big factor with small digits

$$t \approx \frac{d_1}{B_g} + \frac{d_2}{B_g^2} + \ldots + \frac{d_\ell}{B_g^\ell}, \qquad d_i \in \left[-\frac{B_g}{2}, \frac{B_g}{2}\right)$$

with error at most $\varepsilon = 1/(2B_g^\ell)$.

Here: $B_g = 2^7$, $\ell = 3$, digits in $[-64, 64)$, $\varepsilon = 2^{-22}$.

**The same idea in base 10,** $\ell = 2$, digits in $[-5, 5)$




check: $-\frac{3}{10} + \frac{3}{100} = -0.27 \equiv 0.73$,
error $0.0016 \leq \varepsilon = 0.005$
implementation trick (later): $t + 0.555 \equiv 0.2866$,
digits 2, 8 minus 5 $\Rightarrow -3, 3$: one addition!

# Step 2: precomputed encryptions (TRGSW)

**Goal:** an encryption of $m \cdot \varphi(d) = m\,b - m\,s'\,a$, for $d = (a, b)$ and a secret bit $m$.

# Step 2: precomputed encryptions (TRGSW)

**Goal:** an encryption of $m \cdot \varphi(d) = m\,b - m\,s'\,a$, for $d = (a, b)$ and a secret bit $m$.

Decompose $a \approx \sum u_i B_g^{-i}$ and $b \approx \sum w_i B_g^{-i}$: the digits are public and small, the server computes them.

$$m\,\varphi(d) \approx \sum_i w_i \cdot \left(\frac{m}{B_g^i}\right) + \sum_i u_i \cdot \left(\frac{-m\,s'}{B_g^i}\right)$$

# Step 2: precomputed encryptions (TRGSW)

**Goal:** an encryption of $m \cdot \varphi(d) = m\,b - m\,s'\,a$, for $d = (a, b)$ and a secret bit $m$.

Decompose $a \approx \sum u_i B_g^{-i}$ and $b \approx \sum w_i B_g^{-i}$: the digits are public and small, the server computes them.

$$m\,\varphi(d) \approx \sum_i w_i \cdot \left(\frac{m}{B_g^i}\right) + \sum_i u_i \cdot \left(\frac{-m\,s'}{B_g^i}\right)$$

The highlighted terms depend on the secret: the server receives them encrypted.

# Step 2: precomputed encryptions (TRGSW)

**Goal:** an encryption of $m \cdot \varphi(d) = m\,b - m\,s'\,a$, for $d = (a, b)$ and a secret bit $m$.

Decompose $a \approx \sum u_i B_g^{-i}$ and $b \approx \sum w_i B_g^{-i}$: the digits are public and small, the server computes them.

$$m\,\varphi(d) \approx \sum_i w_i \cdot \left(\frac{m}{B_g^i}\right) + \sum_i u_i \cdot \left(\frac{-m\,s'}{B_g^i}\right)$$

The highlighted terms depend on the secret: the server receives them encrypted.

**TRGSW sample of $m$**

$2\ell$ TRLWE encryptions of zero, plus $m/B_g^i$ in one component:
row $i$ has phase $e_i - m\,s'/B_g^i$, row $\ell + i$ has phase $e_{\ell+i} + m/B_g^i$. Each row is encrypted: nothing about $m$ leaks.

# Step 3: the external product

| | digit | | first component | second component | phase of the row |
|---|---|---|---|---|---|
| digits of $a$ | $u_1$ | $\times$ | $a_1 + \frac{m}{B_g^1}$ | $b_1$ | $e_1 - m\,s'/B_g^1$ |
| | $u_2$ | $\times$ | $a_2 + \frac{m}{B_g^2}$ | $b_2$ | $e_2 - m\,s'/B_g^2$ |
| | $u_3$ | $\times$ | $a_3 + \frac{m}{B_g^3}$ | $b_3$ | $e_3 - m\,s'/B_g^3$ |
| digits of $b$ | $w_1$ | $\times$ | $a_4$ | $b_4 + \frac{m}{B_g^1}$ | $e_4 + m/B_g^1$ |
| | $w_2$ | $\times$ | $a_5$ | $b_5 + \frac{m}{B_g^2}$ | $e_5 + m/B_g^2$ |
| | $w_3$ | $\times$ | $a_6$ | $b_6 + \frac{m}{B_g^3}$ | $e_6 + m/B_g^3$ |

$\sum$ of the weighted rows: phase $m\Big(\underbrace{\sum_i w_i B_g^{-i}}_{\approx b} - s'\underbrace{\sum_i u_i B_g^{-i}}_{\approx a}\Big) + \underbrace{\sum_i (u_i e_i + w_i e_{3+i})}_{\text{small: digits} \le 64}$

Weight row $i$ by its digit and add: $C \boxdot d \approx$ encryption of $m \cdot \varphi(d)$.

No torus $\times$ torus: obstacle 1 gone.

# Step 3: the external product

| | digit | | first component | second component | phase of the row |
|---|---|---|---|---|---|
| digits of $a$ | $u_1$ | $\times$ | $a_1 + \frac{m}{B_g^1}$ | $b_1$ | $e_1 - m\,s'/B_g^1$ |
| | $u_2$ | $\times$ | $a_2 + \frac{m}{B_g^2}$ | $b_2$ | $e_2 - m\,s'/B_g^2$ |
| | $u_3$ | $\times$ | $a_3 + \frac{m}{B_g^3}$ | $b_3$ | $e_3 - m\,s'/B_g^3$ |
| digits of $b$ | $w_1$ | $\times$ | $a_4$ | $b_4 + \frac{m}{B_g^1}$ | $e_4 + m/B_g^1$ |
| | $w_2$ | $\times$ | $a_5$ | $b_5 + \frac{m}{B_g^2}$ | $e_5 + m/B_g^2$ |
| | $w_3$ | $\times$ | $a_6$ | $b_6 + \frac{m}{B_g^3}$ | $e_6 + m/B_g^3$ |

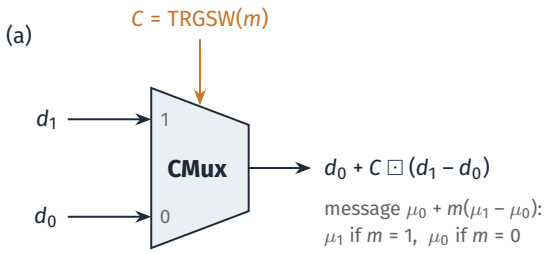
$\sum$ of the weighted rows: phase $m\Big(\underbrace{\sum_i w_i B_g^{-i}}_{\approx b} - s'\underbrace{\sum_i u_i B_g^{-i}}_{\approx a}\Big) + \underbrace{\sum_i (u_i e_i + w_i e_{3+i})}_{\text{small: digits} \le 64}$

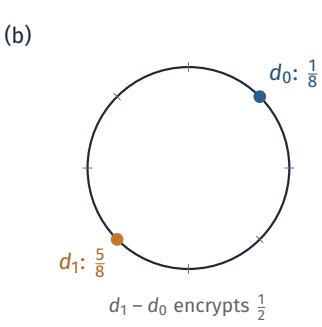
Weight row $i$ by its digit and add: $C \boxdot d \approx$ encryption of $m \cdot \varphi(d)$.
No torus $\times$ torus: obstacle 1 gone.

**Noise grows additively**

$\mathrm{Var} \le 2\ell N \beta^2 \vartheta_C + m\big((1+N)\varepsilon^2 + \vartheta_d\big)$

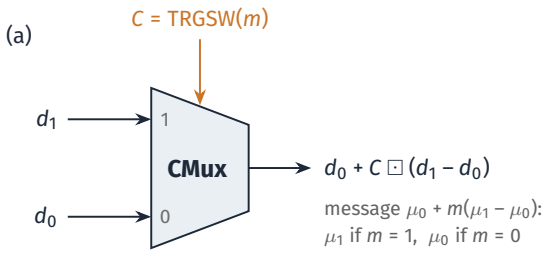
# CMux: an encrypted if-then-else



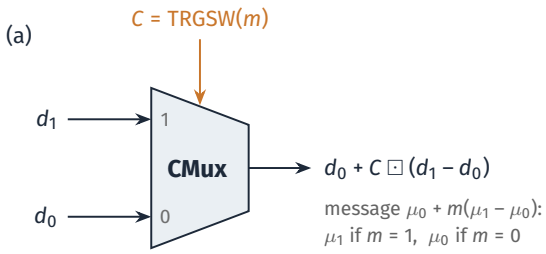

- One external product, the same work for $m = 0$ and $m = 1$.

# CMux: an encrypted if-then-else



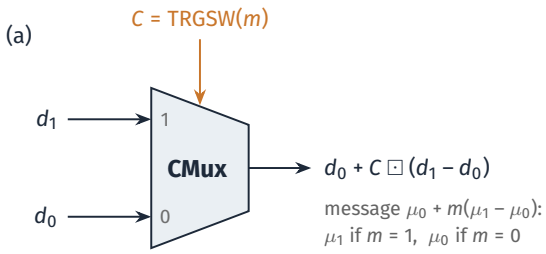

- One external product, the same work for $m = 0$ and $m = 1$.
- $d_1 - d_0$ encrypts $\frac{1}{2}$; $C \boxdot (d_1 - d_0)$ encrypts $m \cdot \frac{1}{2}$.

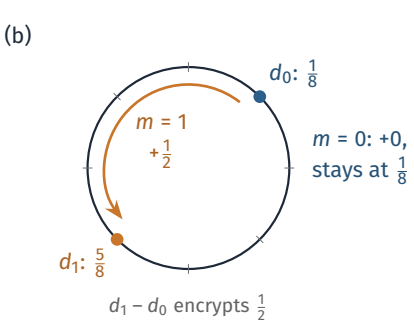
# CMux: an encrypted if-then-else




- One external product, the same work for $m = 0$ and $m = 1$.
- $d_1 - d_0$ encrypts $\frac{1}{2}$; $C \boxdot (d_1 - d_0)$ encrypts $m \cdot \frac{1}{2}$.

# CMux: an encrypted if-then-else




- One external product, the same work for $m = 0$ and $m = 1$.
- $d_1 - d_0$ encrypts $\frac{1}{2}$; $C \boxdot (d_1 - d_0)$ encrypts $m \cdot \frac{1}{2}$.

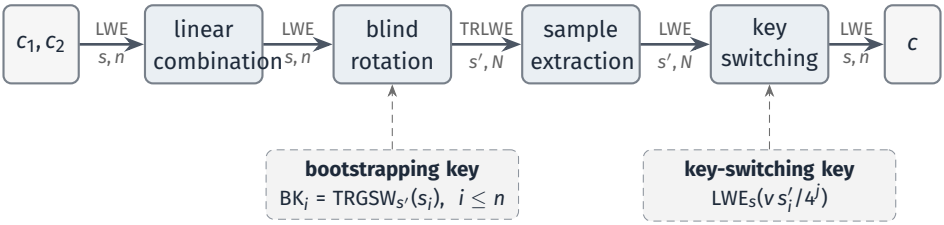
**Exactly the step the plan needs**

$d_1 = X^{k_i}\text{ACC}$, $d_0 = \text{ACC}$, $C = \text{TRGSW}(s_i)$

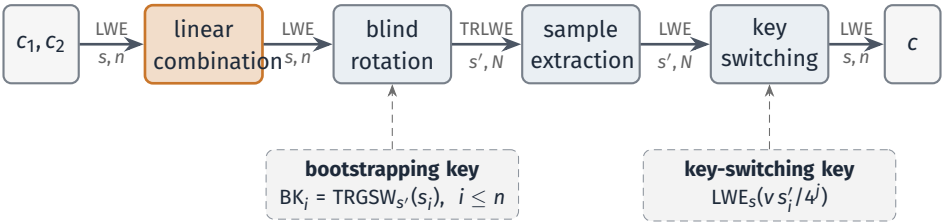
(`test07` checks every key bit this way)

# Gate bootstrapping

# Putting it together: one bootstrapped gate

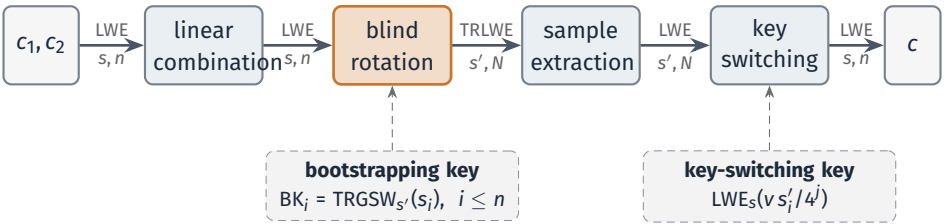


# Putting it together: one bootstrapped gate



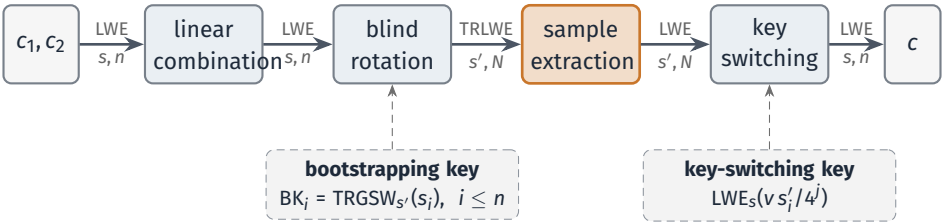

Combine the inputs so that the phase is positive $\iff$ the output is true. Cheap, but noises add up.

# Putting it together: one bootstrapped gate



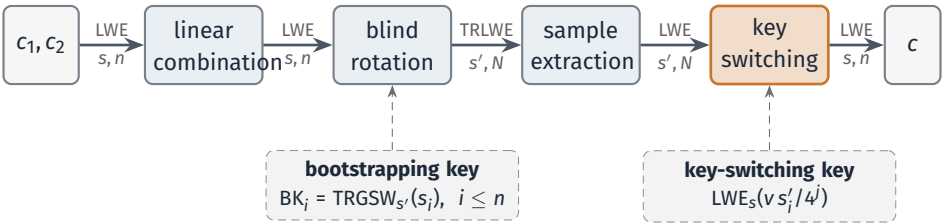

Rotate a test polynomial by the *encrypted* phase: the gate is evaluated and the noise is fresh. > 95% of the time.

# Putting it together: one bootstrapped gate




Read the constant coefficient of the TRLWE as an LWE sample under $s'$. Free, no noise.

# Putting it together: one bootstrapped gate




Bring the result back to key $s$ and dimension $n$, ready for the next gate.

# Blind rotation: the idea

1. Round to $\mathbb{Z}_{2N}$: $\bar{b} = \lfloor 2Nb \rceil$, $\bar{a}_i = \lfloor 2Na_i \rceil$, $\bar{\varphi} = \bar{b} - \sum_i \bar{a}_i s_i$.

# Blind rotation: the idea

1. Round to $\mathbb{Z}_{2N}$: $\bar{b} = \lfloor 2Nb \rceil$, $\bar{a}_i = \lfloor 2Na_i \rceil$, $\bar{\varphi} = \bar{b} - \sum_i \bar{a}_i s_i$.
2. Split the rotation:

$$X^{-\bar{\varphi}} = X^{-\bar{b}} \prod_{i=1}^{n} X^{\bar{a}_i s_i}$$

each factor is $X^{\bar{a}_i}$ or 1 by a secret bit.

# Blind rotation: the idea

1. Round to $\mathbb{Z}_{2N}$: $\bar{b} = \lfloor 2Nb \rceil$, $\bar{a}_i = \lfloor 2Na_i \rceil$, $\bar{\varphi} = \bar{b} - \sum_i \bar{a}_i s_i$.
2. Split the rotation:

$$X^{-\bar{\varphi}} = X^{-\bar{b}} \prod_{i=1}^{n} X^{\bar{a}_i s_i}$$

   each factor is $X^{\bar{a}_i}$ or 1 by a secret bit.
3. $\mathrm{ACC} \leftarrow (0, X^{-\bar{b}} v)$, then for each $i$, a CMux:

$$\mathrm{ACC} \leftarrow \mathrm{ACC} + \mathrm{BK}_i \boxdot \left( (X^{\bar{a}_i} - 1)\, \mathrm{ACC} \right)$$

# Blind rotation: the idea

1. Round to $\mathbb{Z}_{2N}$: $\bar{b} = \lfloor 2Nb \rceil$, $\bar{a}_i = \lfloor 2Na_i \rceil$, $\bar{\varphi} = \bar{b} - \sum_i \bar{a}_i s_i$.
2. Split the rotation:

$$X^{-\bar{\varphi}} = X^{-\bar{b}} \prod_{i=1}^{n} X^{\bar{a}_i s_i}$$

   each factor is $X^{\bar{a}_i}$ or 1 by a secret bit.
3. $\mathrm{ACC} \leftarrow (0, X^{-\bar{b}} v)$, then for each $i$, a CMux:

$$\mathrm{ACC} \leftarrow \mathrm{ACC} + \mathrm{BK}_i \boxdot \left( (X^{\bar{a}_i} - 1)\,\mathrm{ACC} \right)$$

**Why "blind"**

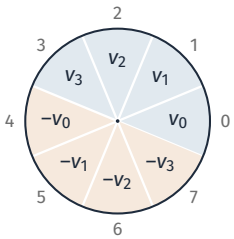
The server knows every $\bar{a}_i$ and $\bar{b}$, but not $s$: it never learns the total rotation.

**Cost and noise**

$n = 630$ CMux = 630 external products.

Fresh noise: depends on BK only.

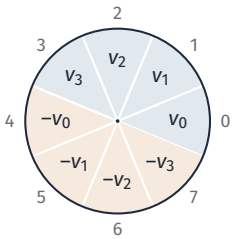
# What are the $v_j$? the entries of the table ($N = 4$)



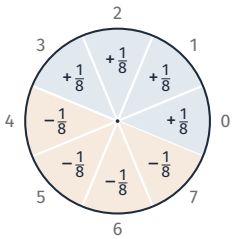

**(a)** any test polynomial
$v = v_0 + v_1 X + v_2 X^2 + v_3 X^3$

- $v_j$ **is the output** for slice $\bar{\varphi} = j$, i.e., for phases close to $j/(2N)$.
- Slice $N + j$ (opposite) returns $-v_j$, because $X^N = -1$.
- To evaluate $f$: $v_j = f\big(j/(2N)\big)$.

# What are the $v_j$? the entries of the table ($N = 4$)




**(a)** any test polynomial
$v = v_0 + v_1X + v_2X^2 + v_3X^3$

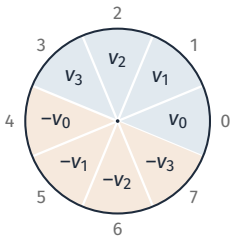



**(b)** gates: all $v_j = \frac{1}{8}$
$v = \frac{1}{8}(1 + X + X^2 + X^3)$

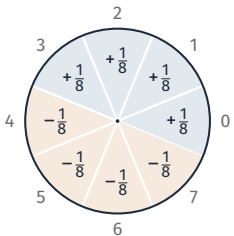
- $v_j$ **is the output** for slice $\bar{\varphi} = j$, i.e., for phases close to $j/(2N)$.
- Slice $N + j$ (opposite) returns $-v_j$, because $X^N = -1$.

- To evaluate $f$: $v_j = f\big(j/(2N)\big)$.
- Gates: all $v_j = \frac{1}{8} \Rightarrow$ sign.

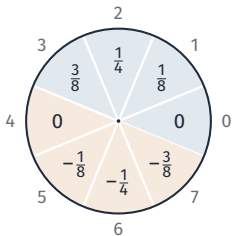
# What are the $v_j$? the entries of the table ($N = 4$)




**(a)** any test polynomial
$v = v_0 + v_1X + v_2X^2 + v_3X^3$




**(b)** gates: all $v_j = \frac{1}{8}$
$v = \frac{1}{8}(1 + X + X^2 + X^3)$



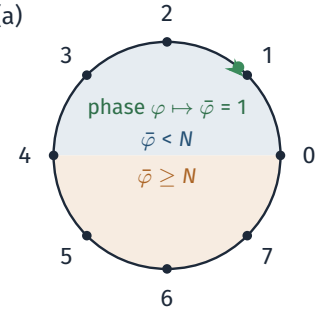

**(c)** $v_j = j/8$
$v = \frac{1}{8}X + \frac{1}{4}X^2 + \frac{3}{8}X^3$

- $v_j$ **is the output** for slice $\bar{\varphi} = j$, i.e., for phases close to $j/(2N)$.
- Slice $N + j$ (opposite) returns $-v_j$, because $X^N = -1$.

- To evaluate $f$: $v_j = f\big(j/(2N)\big)$.
- Gates: all $v_j = \frac{1}{8} \Rightarrow$ sign.
- $v_j = j/8$: returns the rounded phase.

# The test polynomial is a lookup table ($N = 4$)

(a)




$\bar{\varphi} = \lfloor 2N\varphi \rceil,\ 2N = 8$

(b)

| $\bar{\varphi}$ | $X^0$ | $X^1$ | $X^2$ | $X^3$ |
|---|---|---|---|---|
| 0 | $v_0$ | $v_1$ | $v_2$ | $v_3$ |
| 1 | $v_1$ | $v_2$ | $v_3$ | $-v_0$ |
| 2 | $v_2$ | $v_3$ | $-v_0$ | $-v_1$ |
| 3 | $v_3$ | $-v_0$ | $-v_1$ | $-v_2$ |
| 4 | $-v_0$ | $-v_1$ | $-v_2$ | $-v_3$ |
| 5 | $-v_1$ | $-v_2$ | $-v_3$ | $v_0$ |
| 6 | $-v_2$ | $-v_3$ | $v_0$ | $v_1$ |
| 7 | $-v_3$ | $v_0$ | $v_1$ | $v_2$ |

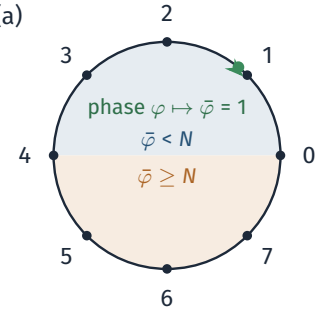
↑ constant coefficient

$X^{-\bar{\varphi}} \cdot v,\ v = v_0 + v_1X + v_2X^2 + v_3X^3$

Rotating by $X^{-\bar{\varphi}}$ brings entry $v_{\bar{\varphi}}$ to the front: each row is the previous one shifted left, and the coefficient that leaves comes back negated ($X^N = -1$).

# The test polynomial is a lookup table ($N = 4$)

(a)




$\bar{\varphi} = \lfloor 2N\varphi \rceil,\ 2N = 8$

(b)

| $\bar{\varphi}$ | $X^0$ | $X^1$ | $X^2$ | $X^3$ | gates: $v_j = \mu$ |
|---|---|---|---|---|---|
| 0 | $v_0$ | $v_1$ | $v_2$ | $v_3$ | $+\mu$ |
| 1 | $v_1$ | $v_2$ | $v_3$ | $-v_0$ | $+\mu$ |
| 2 | $v_2$ | $v_3$ | $-v_0$ | $-v_1$ | $+\mu$ |
| 3 | $v_3$ | $-v_0$ | $-v_1$ | $-v_2$ | $+\mu$ |
| 4 | $-v_0$ | $-v_1$ | $-v_2$ | $-v_3$ | $-\mu$ |
| 5 | $-v_1$ | $-v_2$ | $-v_3$ | $v_0$ | $-\mu$ |
| 6 | $-v_2$ | $-v_3$ | $v_0$ | $v_1$ | $-\mu$ |
| 7 | $-v_3$ | $v_0$ | $v_1$ | $v_2$ | $-\mu$ |

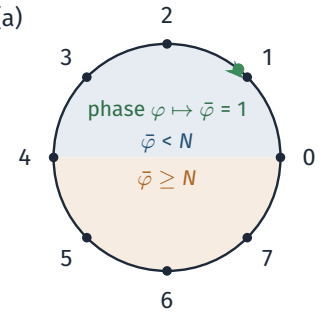
↑ constant coefficient

$X^{-\bar{\varphi}} \cdot v,\ v = v_0 + v_1X + v_2X^2 + v_3X^3$

Rotating by $X^{-\bar{\varphi}}$ brings entry $v_{\bar{\varphi}}$ to the front: each row is the previous one shifted left, and the coefficient that leaves comes back negated ($X^N = -1$).

# The test polynomial is a lookup table ($N = 4$)

(a)




$\bar{\varphi} = \lfloor 2N\varphi \rceil$, $2N = 8$

(b)

| $\bar{\varphi}$ | $X^0$ | $X^1$ | $X^2$ | $X^3$ | gates: $v_j = \mu$ |
|---|---|---|---|---|---|
| 0 | $v_0$ | $v_1$ | $v_2$ | $v_3$ | $+\mu$ |
| 1 | $v_1$ | $v_2$ | $v_3$ | $-v_0$ | $+\mu$ |
| 2 | $v_2$ | $v_3$ | $-v_0$ | $-v_1$ | $+\mu$ |
| 3 | $v_3$ | $-v_0$ | $-v_1$ | $-v_2$ | $+\mu$ |
| 4 | $-v_0$ | $-v_1$ | $-v_2$ | $-v_3$ | $-\mu$ |
| 5 | $-v_1$ | $-v_2$ | $-v_3$ | $v_0$ | $-\mu$ |
| 6 | $-v_2$ | $-v_3$ | $v_0$ | $v_1$ | $-\mu$ |
| 7 | $-v_3$ | $v_0$ | $v_1$ | $v_2$ | $-\mu$ |

↑ constant coefficient

$X^{-\bar{\varphi}} \cdot v$, $v = v_0 + v_1X + v_2X^2 + v_3X^3$

Rotating by $X^{-\bar{\varphi}}$ brings entry $v_{\bar{\varphi}}$ to the front: each row is the previous one shifted left, and the coefficient that leaves comes back negated ($X^N = -1$). Other $v \Rightarrow$ other functions (programmable bootstrapping; `test08` evaluates a 4-entry table).

# How the test polynomial is built, and is it encrypted?

**Built by the server**, from the function:
coefficient $j$ = answer for slice $\bar{\varphi} = j$.

- Gates: all $N = 1024$ coefficients
  $= \mu = \frac{1}{8}$ (`0x20000000`):

```
TorusPoly testv(N, MU);
```

# How the test polynomial is built, and is it encrypted?

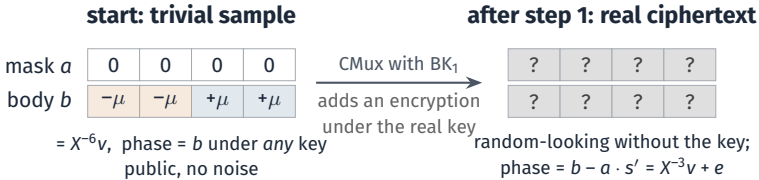
**Built by the server**, from the function: coefficient $j$ = answer for slice $\bar{\varphi} = j$.

- Gates: all $N = 1024$ coefficients $= \mu = \frac{1}{8}$ (`0x20000000`):

```
TorusPoly testv(N, MU);
```

**Not encrypted: it is public**

The function is no secret ("compute a NAND"). What is secret is the input, i.e., *which* entry gets selected.



# How the test polynomial is built, and is it encrypted?

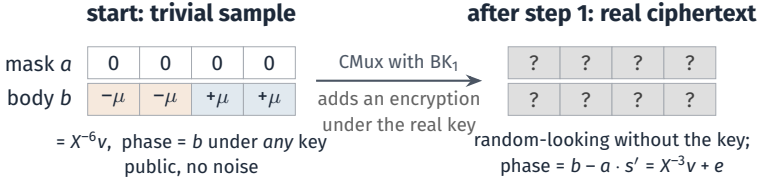
**Built by the server**, from the function: coefficient $j$ = answer for slice $\bar{\varphi} = j$.

- Gates: all $N = 1024$ coefficients $= \mu = \frac{1}{8}$ (`0x20000000`):

```
TorusPoly testv(N, MU);
```

**Not encrypted: it is public**

The function is no secret ("compute a NAND"). What is secret is the input, i.e., *which* entry gets selected.




Start: **trivial** sample $(0, X^{-\bar{b}}v)$, public since $\bar{b}$ is public. After the first CMux: a real ciphertext $\Rightarrow$ **public table, secret selection**.

# Blind rotation, step by step ($n = 3$, $N = 4$)

**Secret:** $s = (1, 0, 1)$

**Public:** $\bar{b} = 6$, $\bar{\mathbf{a}} = (3, 5, 2)$

**Goal:** $v_{\bar{\varphi}}$ without knowing $\bar{\varphi}$

start: $X^{-\bar{b}}v = X^{-6}v$

| $-v_2$ | $-v_3$ | $v_0$ | $v_1$ |
|---|---|---|---|

# Blind rotation, step by step ($n = 3, N = 4$)

**Secret:** $s = (1, 0, 1)$

**Public:** $\bar{b} = 6$, $\bar{\mathbf{a}} = (3, 5, 2)$

**Goal:** $v_{\bar{\varphi}}$ without knowing $\bar{\varphi}$

start: $X^{-\bar{b}}v = X^{-6}v$



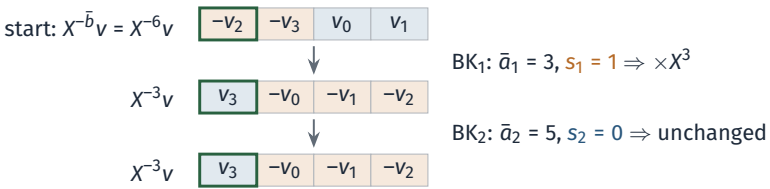

$\mathrm{BK}_1$: $\bar{a}_1 = 3$, $s_1 = 1 \Rightarrow \times X^3$

# Blind rotation, step by step ($n = 3, N = 4$)

**Secret:** $s = (1, 0, 1)$

**Public:** $\bar{b} = 6$, $\bar{\mathbf{a}} = (3, 5, 2)$

**Goal:** $v_{\bar{\varphi}}$ without knowing $\bar{\varphi}$



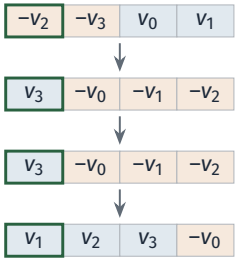
# Blind rotation, step by step ($n = 3, N = 4$)

**Secret:** $s = (1, 0, 1)$

**Public:** $\bar{b} = 6$, $\bar{\mathbf{a}} = (3, 5, 2)$

**Goal:** $v_{\bar{\varphi}}$ without knowing $\bar{\varphi}$

start: $X^{-\bar{b}}v = X^{-6}v$




$\mathrm{BK}_1$: $\bar{a}_1 = 3$, $s_1 = 1 \Rightarrow \times X^3$

$\mathrm{BK}_2$: $\bar{a}_2 = 5$, $s_2 = 0 \Rightarrow$ unchanged

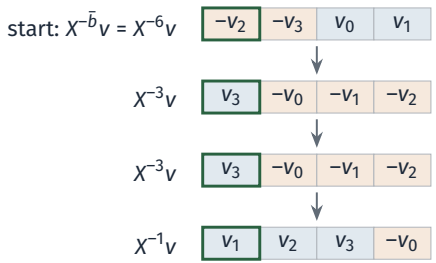
$\mathrm{BK}_3$: $\bar{a}_3 = 2$, $s_3 = 1 \Rightarrow \times X^2$

# Blind rotation, step by step ($n = 3$, $N = 4$)

**Secret:** $s = (1, 0, 1)$

**Public:** $\bar{b} = 6$, $\bar{\mathbf{a}} = (3, 5, 2)$

**Goal:** $v_{\bar{\varphi}}$ without knowing $\bar{\varphi}$




$\mathrm{BK}_1$: $\bar{a}_1 = 3$, $s_1 = 1 \Rightarrow \times X^3$

$\mathrm{BK}_2$: $\bar{a}_2 = 5$, $s_2 = 0 \Rightarrow$ unchanged

$\mathrm{BK}_3$: $\bar{a}_3 = 2$, $s_3 = 1 \Rightarrow \times X^2$

output $v_1$, as $\bar{\varphi} = 6 - 3 - 2 = 1$

# The same run with numbers: bootstrapping an encryption of true

**Input:** LWE of true ($+\frac{1}{8}$),
$s = (1, 0, 1)$, error $e = 0.003$

$\mathbf{a} = (0.37,\ 0.61,\ 0.24)$

$b = 0.37 + 0.24 + 0.125 + 0.003 = 0.738$

# The same run with numbers: bootstrapping an encryption of true

**Input:** LWE of true ($+\frac{1}{8}$),
$s = (1, 0, 1)$, error $e = 0.003$

$\mathbf{a} = (0.37,\ 0.61,\ 0.24)$

$b = 0.37 + 0.24 + 0.125 + 0.003 = 0.738$

**Modulus switching** ($\times 8$, round):

$\bar{b} = \lfloor 5.904 \rceil = 6$

$\bar{\mathbf{a}} = (\lfloor 2.96 \rceil, \lfloor 4.88 \rceil, \lfloor 1.92 \rceil) = (3, 5, 2)$

$\bar{\varphi} = 6 - 3 - 2 = 1$ (unknown to the server)

# The same run with numbers: bootstrapping an encryption of true

**Input:** LWE of true ($+\frac{1}{8}$),
$s = (1, 0, 1)$, error $e = 0.003$

$\mathbf{a} = (0.37,\ 0.61,\ 0.24)$

$b = 0.37 + 0.24 + 0.125 + 0.003 = 0.738$

**Modulus switching** ($\times 8$, round):

$\bar{b} = \lfloor 5.904 \rceil = 6$

$\bar{\mathbf{a}} = (\lfloor 2.96 \rceil, \lfloor 4.88 \rceil, \lfloor 1.92 \rceil) = (3, 5, 2)$

$\bar{\varphi} = 6 - 3 - 2 = 1$ (unknown to the server)

**Gate test polynomial:** $v = [\mu, \mu, \mu, \mu]$

| step | $\bar{a}_i$, $s_i$ | message of ACC | coefficients |
|---|---|---|---|
| start | – | $X^{-6}v$ (trivial) | $[-\mu, -\mu, +\mu, +\mu]$ |

$X^{-6} = X^2$: $X^2 v = \mu(X^2 + X^3 + X^4 + X^5) = \mu(-1 - X + X^2 + X^3)$

# The same run with numbers: bootstrapping an encryption of true

**Input:** LWE of true ($+\frac{1}{8}$),
$s = (1, 0, 1)$, error $e = 0.003$

$\mathbf{a} = (0.37,\ 0.61,\ 0.24)$

$b = 0.37 + 0.24 + 0.125 + 0.003 = 0.738$

**Modulus switching** ($\times 8$, round):

$\bar{b} = \lfloor 5.904 \rceil = 6$

$\bar{\mathbf{a}} = (\lfloor 2.96 \rceil, \lfloor 4.88 \rceil, \lfloor 1.92 \rceil) = (3, 5, 2)$

$\bar{\varphi} = 6 - 3 - 2 = 1$ (unknown to the server)

**Gate test polynomial:** $v = [\mu, \mu, \mu, \mu]$

| step | $\bar{a}_i$, $s_i$ | message of ACC | coefficients |
|---|---|---|---|
| start | – | $X^{-6}v$ (trivial) | $[-\mu, -\mu, +\mu, +\mu]$ |
| $i = 1$ | 3, 1 | $X^{-3}v$ | $[+\mu, -\mu, -\mu, -\mu]$ |

$X^{-6} = X^2$: $X^2 v = \mu(X^2 + X^3 + X^4 + X^5) = \mu(-1 - X + X^2 + X^3)$

# The same run with numbers: bootstrapping an encryption of true

**Input:** LWE of true ($+\frac{1}{8}$),
$s = (1, 0, 1)$, error $e = 0.003$

$\mathbf{a} = (0.37,\ 0.61,\ 0.24)$

$b = 0.37 + 0.24 + 0.125 + 0.003 = 0.738$

**Modulus switching** ($\times 8$, round):

$\bar{b} = \lfloor 5.904 \rceil = 6$

$\bar{\mathbf{a}} = (\lfloor 2.96 \rceil, \lfloor 4.88 \rceil, \lfloor 1.92 \rceil) = (3, 5, 2)$

$\bar{\varphi} = 6 - 3 - 2 = 1$ (unknown to the server)

**Gate test polynomial:** $v = [\mu, \mu, \mu, \mu]$

| step | $\bar{a}_i$, $s_i$ | message of ACC | coefficients |
|---|---|---|---|
| start | – | $X^{-6}v$ (trivial) | $[-\mu, -\mu, +\mu, +\mu]$ |
| $i = 1$ | 3, 1 | $X^{-3}v$ | $[+\mu, -\mu, -\mu, -\mu]$ |
| $i = 2$ | 5, 0 | $X^{-3}v$ | $[+\mu, -\mu, -\mu, -\mu]$ |

$X^{-6} = X^2$: $X^2 v = \mu(X^2 + X^3 + X^4 + X^5) = \mu(-1 - X + X^2 + X^3)$

# The same run with numbers: bootstrapping an encryption of true

**Input:** LWE of true ($+\frac{1}{8}$),
$s = (1, 0, 1)$, error $e = 0.003$

$\mathbf{a} = (0.37,\ 0.61,\ 0.24)$

$b = 0.37 + 0.24 + 0.125 + 0.003 = 0.738$

**Modulus switching** ($\times 8$, round):

$\bar{b} = \lfloor 5.904 \rceil = 6$

$\bar{\mathbf{a}} = (\lfloor 2.96 \rceil, \lfloor 4.88 \rceil, \lfloor 1.92 \rceil) = (3, 5, 2)$

$\bar{\varphi} = 6 - 3 - 2 = 1$ (unknown to the server)

**Gate test polynomial:** $v = [\mu, \mu, \mu, \mu]$

| step | $\bar{a}_i$, $s_i$ | message of ACC | coefficients |
|---|---|---|---|
| start | – | $X^{-6}v$ (trivial) | $[-\mu, -\mu, +\mu, +\mu]$ |
| $i = 1$ | 3, 1 | $X^{-3}v$ | $[+\mu, -\mu, -\mu, -\mu]$ |
| $i = 2$ | 5, 0 | $X^{-3}v$ | $[+\mu, -\mu, -\mu, -\mu]$ |
| $i = 3$ | 2, 1 | $X^{-1}v$ | $[+\mu, +\mu, +\mu, -\mu]$ |

$X^{-6} = X^2$: $X^2 v = \mu(X^2 + X^3 + X^4 + X^5) = \mu(-1 - X + X^2 + X^3)$

# The same run with numbers: bootstrapping an encryption of true

**Input:** LWE of true ($+\frac{1}{8}$),
$s = (1, 0, 1)$, error $e = 0.003$

$\mathbf{a} = (0.37,\ 0.61,\ 0.24)$

$b = 0.37 + 0.24 + 0.125 + 0.003 = 0.738$

**Modulus switching** ($\times 8$, round):

$\bar{b} = \lfloor 5.904 \rceil = 6$

$\bar{\mathbf{a}} = (\lfloor 2.96 \rceil, \lfloor 4.88 \rceil, \lfloor 1.92 \rceil) = (3, 5, 2)$

$\bar{\varphi} = 6 - 3 - 2 = 1$ (unknown to the server)

**Gate test polynomial:** $v = [\mu, \mu, \mu, \mu]$

| step | $\bar{a}_i$, $s_i$ | message of ACC | coefficients |
|---|---|---|---|
| start | – | $X^{-6}v$ (trivial) | $[-\mu, -\mu, +\mu, +\mu]$ |
| $i = 1$ | 3, 1 | $X^{-3}v$ | $[+\mu, -\mu, -\mu, -\mu]$ |
| $i = 2$ | 5, 0 | $X^{-3}v$ | $[+\mu, -\mu, -\mu, -\mu]$ |
| $i = 3$ | 2, 1 | $X^{-1}v$ | $[+\mu, +\mu, +\mu, -\mu]$ |

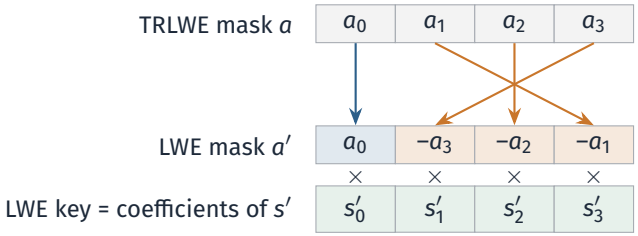
$X^{-6} = X^2$: $X^2 v = \mu(X^2 + X^3 + X^4 + X^5) = \mu(-1 - X + X^2 + X^3)$

## Sample extraction

constant coefficient $+\mu = +\frac{1}{8} \Rightarrow$ true, with fresh noise.

any $\bar{\varphi} \in \{0, .., 3\}$ gives $+\mu$; $\{4, .., 7\}$ gives $-\mu$

# Sample extraction ($N = 4$)




$a \cdot s'$ has constant coefficient
$a_0 s'_0 - a_3 s'_1 - a_2 s'_2 - a_1 s'_3$
(degrees adding up to 4 = $N$ wrap around with a sign flip: $X^4 = -1$)

- **Why:** blind rotation gives a TRLWE (a whole table); the next gate needs one LWE sample of position 0.
- Reorder and negate the mask: an LWE sample of dimension $N$, under the coefficients of $s'$. No arithmetic, no noise.

# Key switching: from $s'$ ($N = 1024$) back to $s$ ($n = 630$)

| | | $v = 0$ | $v = 1$ | $v = 2$ | $v = 3$ |
|---|---|---|---|---|---|
| $a_{i,1} = 2$ | level $j = 1$ | | | subtract | |
| $a_{i,2} = 3$ | level $j = 2$ | | | | subtract |
| $a_{i,3} = 0$ | level $j = 3$ | skip | | | |
| $a_{i,4} = 3$ | level $j = 4$ | | | | subtract |
| $a_{i,5} = 0$ | level $j = 5$ | skip | | | |
| $a_{i,6} = 3$ | level $j = 6$ | | | | subtract |
| $a_{i,7} = 0$ | level $j = 7$ | skip | | | |
| $a_{i,8} = 3$ | level $j = 8$ | | | | subtract |

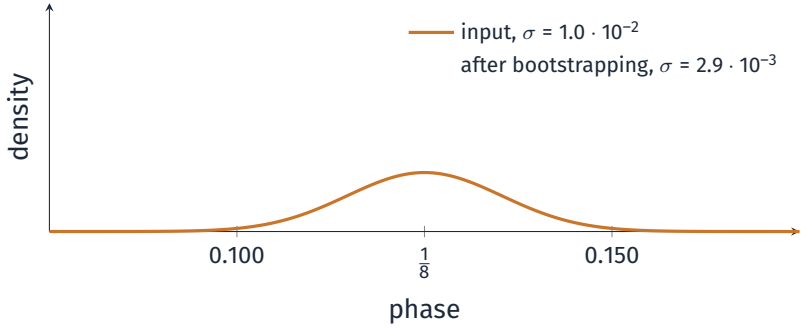
entry $(i, j, v)$:
$\mathrm{LWE}_s\left(v\, s'_i / 4^j\right)$

$a_i = 0.7 = \texttt{0xB3333333}$
in base 4: 0.23030303 . . .

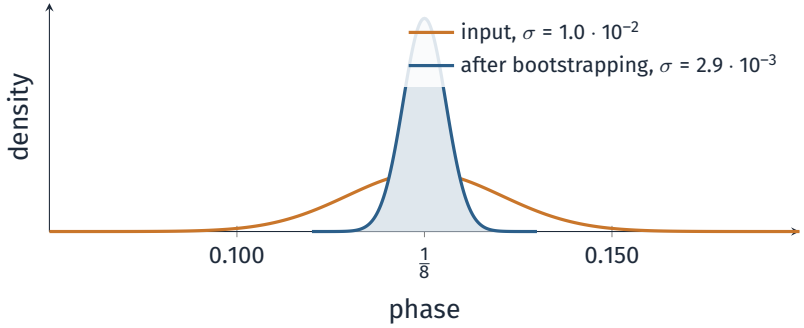
$\Rightarrow$ 5 subtractions, no products

- **Why:** the extracted sample is under $s'$ (size 1024); gates expect $s$ (size 630).
- The base-4 digits of each $a_i$ select table entries.
- $c' = (\mathbf{0}, b) - \sum_{i,j} \mathrm{KSK}_{i,j,a_{ij}}$: same phase, now under $s$.
- No multiplications: memory (79 MiB) for time.

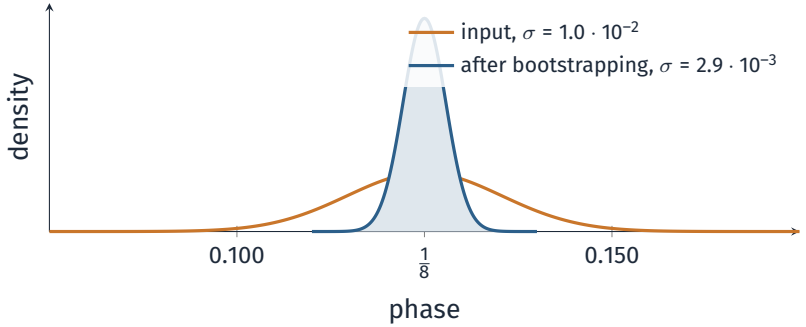
# Bootstrapping resets the noise



# Bootstrapping resets the noise



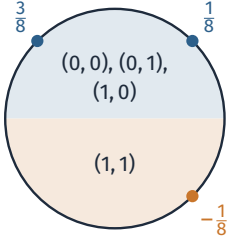
# Bootstrapping resets the noise



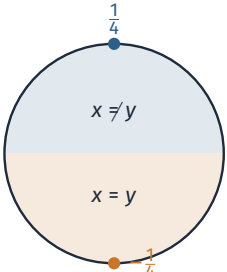

**The key property**

The output noise depends only on the evaluation key, **not** on the input noise ($\vartheta_{\text{out}} \leq 2.2 \cdot 10^{-5}$): gates can be chained forever. (`test09`: inputs 300× noisier than fresh ones)

# Homomorphic gates




**NAND**

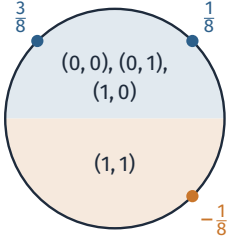



**XOR**

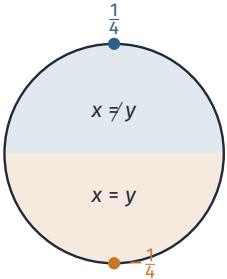
Bits are $\pm\frac{1}{8}$. A gate = linear combination whose phase is positive iff the output is true, then bootstrap:

| Gate | Combination | Margin |
|---|---|---|
| NAND | $(\mathbf{0}, \frac{1}{8}) - c_x - c_y$ | $\frac{1}{8}$ |
| AND | $(\mathbf{0}, -\frac{1}{8}) + c_x + c_y$ | $\frac{1}{8}$ |
| OR | $(\mathbf{0}, \frac{1}{8}) + c_x + c_y$ | $\frac{1}{8}$ |
| XOR | $(\mathbf{0}, \frac{1}{4}) + 2(c_x + c_y)$ | $\frac{1}{4}$ |
| NOT | $-c_x$ (free) | — |

# Homomorphic gates




**NAND**




**XOR**

Bits are $\pm\frac{1}{8}$. A gate = linear combination whose phase is positive iff the output is true, then bootstrap:

| Gate | Combination | Margin |
|---|---|---|
| NAND | $(\mathbf{0}, \frac{1}{8}) - c_x - c_y$ | $\frac{1}{8}$ |
| AND | $(\mathbf{0}, -\frac{1}{8}) + c_x + c_y$ | $\frac{1}{8}$ |
| OR | $(\mathbf{0}, \frac{1}{8}) + c_x + c_y$ | $\frac{1}{8}$ |
| XOR | $(\mathbf{0}, \frac{1}{4}) + 2(c_x + c_y)$ | $\frac{1}{4}$ |
| NOT | $-c_x$ (free) | — |

**XOR trick:** doubling sends both equal-input cases to $\pm\frac{1}{2}$, the same torus point.

# MUX with a single key switching

$\mathrm{MUX}(s, x, y) = (s \wedge x) \vee (\neg s \wedge y)$

- Naively: 3 bootstrappings.
- The two ANDs are never both true $\Rightarrow$ OR = sum.

# MUX with a single key switching

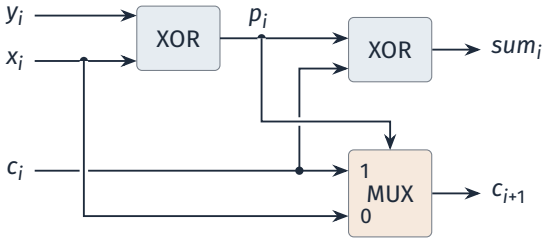
$\mathrm{MUX}(s, x, y) = (s \wedge x) \vee (\neg s \wedge y)$

- Naively: 3 bootstrappings.
- The two ANDs are never both true $\Rightarrow$ OR = sum.
- 2 blind rotations (no key switching), add, $+\frac{1}{8}$, one key switching.

| $s$ | $x$ | $y$ | $u_1$ | $u_2$ | $u_1+u_2+\frac{1}{8}$ | out |
|---|---|---|---|---|---|---|
| 0 | 0 | 1 | $-\frac{1}{8}$ | $+\frac{1}{8}$ | $+\frac{1}{8}$ | $y$ |
| 0 | 1 | 0 | $-\frac{1}{8}$ | $-\frac{1}{8}$ | $-\frac{1}{8}$ | $y$ |
| 1 | 0 | 1 | $-\frac{1}{8}$ | $-\frac{1}{8}$ | $-\frac{1}{8}$ | $x$ |
| 1 | 1 | 0 | $+\frac{1}{8}$ | $-\frac{1}{8}$ | $+\frac{1}{8}$ | $x$ |

$u_1 = \mathrm{BR}(s \wedge x)$, $u_2 = \mathrm{BR}(\neg s \wedge y)$; all 8 rows in the report

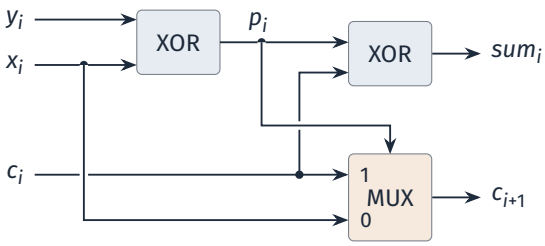
# An encrypted ripple-carry adder




$sum_i = p_i \oplus c_i,\ \ c_{i+1} = \mathrm{MUX}(p_i; c_i, x_i)$

4 blind rotations per bit

# An encrypted ripple-carry adder




$sum_i = p_i \oplus c_i$, $c_{i+1} = \mathrm{MUX}(p_i; c_i, x_i)$

4 blind rotations per bit

**Example:** 5 + 3 in 3 bits (LSB first)

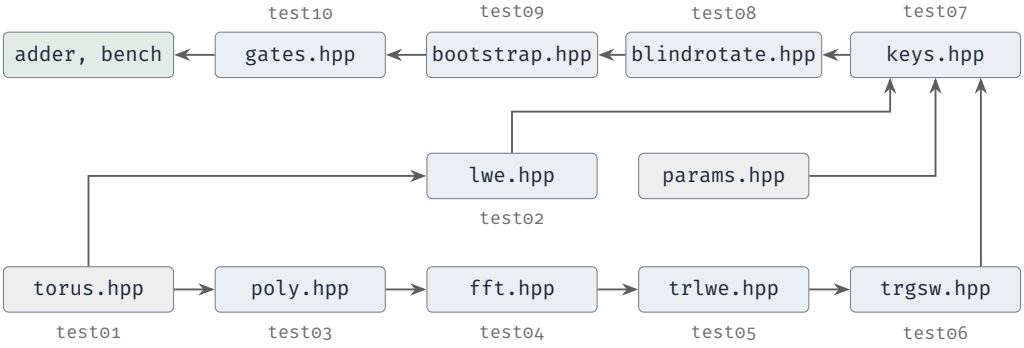
| $i$ | $x_i\, y_i$ | $c_i$ | $p_i$ | $sum_i$ | $c_{i+1}$ |
|---|---|---|---|---|---|
| 0 | 1 1 | 0 | 0 | 0 | $x_0 = 1$ |
| 1 | 0 1 | 1 | 1 | 0 | $c_1 = 1$ |
| 2 | 1 0 | 1 | 1 | 0 | $c_2 = 1$ |

result $1000_2 = 8$ ✓

the server runs the same 9 gates for any input

# Implementation

# Architecture: one header per layer, one test per header




**526** lines
header-only, no dependencies

**10** unit tests
fixed seeds, independent references

**bottom-up**
no layer built on a failing test

# Torus arithmetic in `uint32_t`

Three C++ rules carry the whole arithmetic:

1. **Unsigned arithmetic is mod** $2^{32}$ – exactly the torus. Signed overflow is undefined behavior: an optimizer may break it.

# Torus arithmetic in `uint32_t`

Three C++ rules carry the whole arithmetic:

1. **Unsigned arithmetic is mod** $2^{32}$ – exactly the torus. Signed overflow is undefined behavior: an optimizer may break it.
2. **`k * t` converts `k` to unsigned** – still correct mod $2^{32}$ for negative $k$.

# Torus arithmetic in `uint32_t`

Three C++ rules carry the whole arithmetic:

1. **Unsigned arithmetic is mod** $2^{32}$ – exactly the torus. Signed overflow is undefined behavior: an optimizer may break it.
2. **`k * t` converts `k` to unsigned** – still correct mod $2^{32}$ for negative $k$.
3. **`int32_t(t)` rereads the bits as signed** – the representative in $[-\frac{1}{2}, \frac{1}{2})$: how noise is measured and how a bit is decrypted.

# Torus arithmetic in `uint32_t`

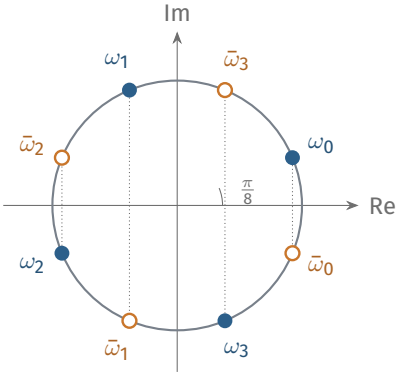
Three C++ rules carry the whole arithmetic:

1. **Unsigned arithmetic is mod $2^{32}$** – exactly the torus. Signed overflow is undefined behavior: an optimizer may break it.
2. **`k * t` converts `k` to unsigned** – still correct mod $2^{32}$ for negative $k$.
3. **`int32_t(t)` rereads the bits as signed** – the representative in $[-\frac{1}{2}, \frac{1}{2})$: how noise is measured and how a bit is decrypted.

```
using Torus = uint32_t;
// decrypt a bit: sign of the phase
bool bit = int32_t(lwePhase(c, s)) > 0;
```

```
// round t to Z_2N: product in 64 bits,
// add 1/2, drop the 32 low bits
(uint64_t(t) * twoN + (1ull << 31)) >> 32
```

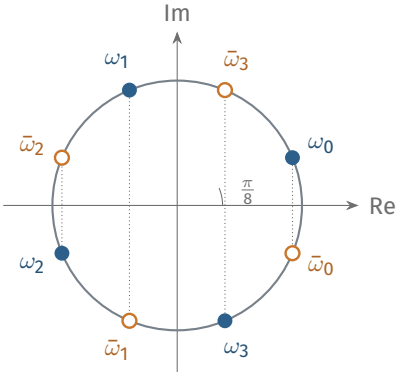
# Why evaluate at the roots of $X^N + 1$?




roots of $X^8 + 1$: we use $\omega_0, \ldots, \omega_3$; the others are their conjugates.

At a root, $\omega^N = -1$: the reduction mod $X^N + 1$ is automatic.

# Why evaluate at the roots of $X^N + 1$?



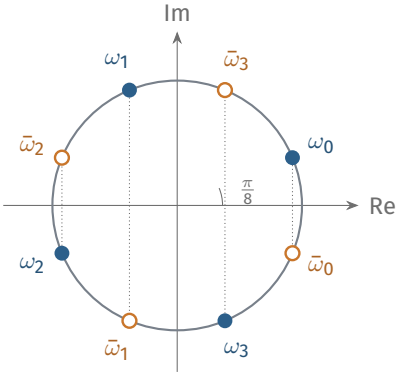

roots of $X^8 + 1$: we use $\omega_0, \ldots, \omega_3$; the others are their conjugates.

At a root, $\omega^N = -1$: the reduction mod $X^N + 1$ is automatic.

**Example** ($N = 4$): $a = 1 + X$, $b = X^3$, $a \cdot b = -1 + X^3$

| | $\omega_0 = e^{i\pi/4}$ | $\omega_1 = e^{i5\pi/4}$ |
|---|---|---|
| $a(\omega)$ | $1.707 + 0.707i$ | $0.293 - 0.707i$ |
| $b(\omega)$ | $-0.707 + 0.707i$ | $0.707 - 0.707i$ |
| $a(\omega)\, b(\omega)$ | $-1.707 + 0.707i$ | $-0.293 - 0.707i$ |

# Why evaluate at the roots of $X^N + 1$?




roots of $X^8 + 1$: we use $\omega_0, \ldots, \omega_3$; the others are their conjugates.

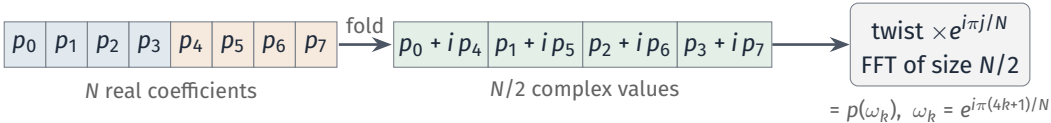
At a root, $\omega^N = -1$: the reduction mod $X^N + 1$ is automatic.

**Example** ($N = 4$): $a = 1 + X$, $b = X^3$, $a \cdot b = -1 + X^3$

| | $\omega_0 = e^{i\pi/4}$ | $\omega_1 = e^{i5\pi/4}$ |
|---|---|---|
| $a(\omega)$ | $1.707 + 0.707i$ | $0.293 - 0.707i$ |
| $b(\omega)$ | $-0.707 + 0.707i$ | $0.707 - 0.707i$ |
| $a(\omega)\, b(\omega)$ | $-1.707 + 0.707i$ | $-0.293 - 0.707i$ |
| $-1 + \omega^3$ | $-1.707 + 0.707i$ | $-0.293 - 0.707i$ |

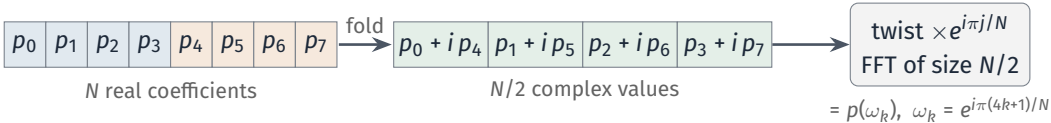
$N$ products instead of $N^2$, plus the cost of the transforms

# Negacyclic FFT: fold and twist




- Evaluate at roots of $X^N + 1$: the reduction happens by itself.
- Real coefficients: one root per conjugate pair suffices.
- Radix-2 Cooley–Tukey, tables precomputed once.
- Hand-written `cmul`: `std::complex` checks NaN/$\infty$.

# Negacyclic FFT: fold and twist




- Evaluate at roots of $X^N + 1$: the reduction happens by itself.
- Real coefficients: one root per conjugate pair suffices.
- Radix-2 Cooley–Tukey, tables precomputed once.
- Hand-written `cmul`: `std::complex` checks NaN/$\infty$.

**Is `double` precise enough?**

digit · torus · $N$ terms:
$2^6 \cdot 2^{31} \cdot 2^{10} = 2^{47} < 2^{53}$

measured: bit-exact, and 60× faster than schoolbook (325 µs → 5.4 µs)

# The offset trick (1/2): balanced digits with one addition

- $2N\ell = 6144$ digits per external product: decomposition must be cheap.
- Unsigned digits $[0, B_g)$ are bit fields, but balanced digits $[-\frac{B_g}{2}, \frac{B_g}{2})$ halve $\beta$: $4\times$ less noise.
- Converting needs carry propagation *and* rounding.

# The offset trick (1/2): balanced digits with one addition

- $2N\ell = 6144$ digits per external product: decomposition must be cheap.
- Unsigned digits $[0, B_g)$ are bit fields, but balanced digits $[-\frac{B_g}{2}, \frac{B_g}{2})$ halve $\beta$: $4\times$ less noise.
- Converting needs carry propagation *and* rounding.

**Lemma (offset trick)**

With $h_i = 2^{32-7i}$ and $\mathit{offset} = \sum_{i=1}^{\ell} \frac{B_g}{2} h_i + \frac{h_\ell}{2}$: take the bit fields $u_i$ of $t + \mathit{offset}$, set $d_i = u_i - \frac{B_g}{2}$. Then

$$t = \sum_i d_i h_i + \underbrace{(r - h_\ell/2)}_{|\cdot| \le \varepsilon} \pmod{2^{32}}.$$

# The offset trick (1/2): balanced digits with one addition

- $2N\ell = 6144$ digits per external product: decomposition must be cheap.
- Unsigned digits $[0, B_g)$ are bit fields, but balanced digits $[-\frac{B_g}{2}, \frac{B_g}{2})$ halve $\beta$: $4\times$ less noise.
- Converting needs carry propagation *and* rounding.

**Lemma (offset trick)**

With $h_i = 2^{32-7i}$ and $\mathit{offset} = \sum_{i=1}^{\ell} \frac{B_g}{2} h_i + \frac{h_\ell}{2}$: take the bit fields $u_i$ of $t + \mathit{offset}$, set $d_i = u_i - \frac{B_g}{2}$. Then

$$t = \sum_i d_i h_i + \underbrace{(r - h_\ell/2)}_{|\cdot| \le \varepsilon} \pmod{2^{32}}.$$

Proof: write $t + \mathit{offset} = \sum_i u_i h_i + r$ and subtract. Base 10: $\mathit{offset} = 0.555$, $0.7316 + 0.555 \equiv 0.2866 \Rightarrow (2, 8) - 5 = (-3, 3)$.

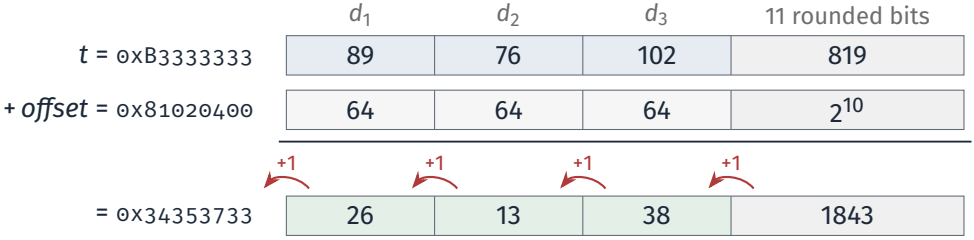
# The offset trick (2/2): $t = 0.7$, $B_g = 2^7$, $\ell = 3$

| | $d_1$ | $d_2$ | $d_3$ | 11 rounded bits |
|---|---|---|---|---|
| $t =$ `0xB3333333` | 89 | 76 | 102 | 819 |

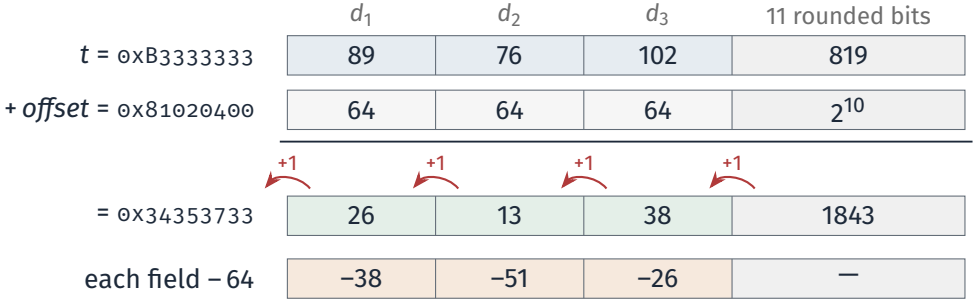
# The offset trick (2/2): $t = 0.7$, $B_g = 2^7$, $\ell = 3$

| | $d_1$ | $d_2$ | $d_3$ | 11 rounded bits |
|---|---|---|---|---|
| $t$ = `0xB3333333` | 89 | 76 | 102 | 819 |
| + *offset* = `0x81020400` | 64 | 64 | 64 | $2^{10}$ |

# The offset trick (2/2): $t = 0.7$, $B_g = 2^7$, $\ell = 3$

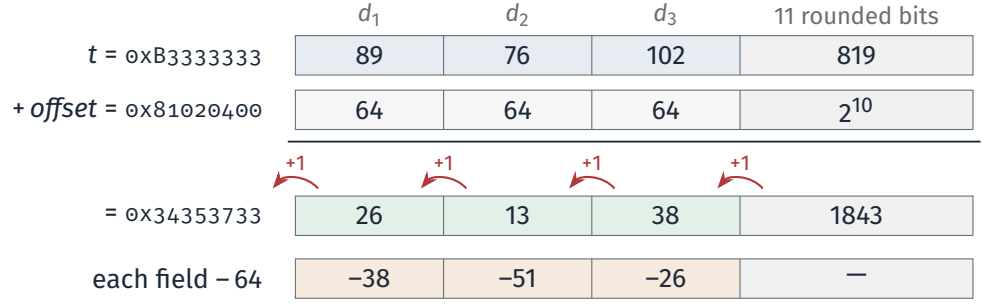


# The offset trick (2/2): $t = 0.7$, $B_g = 2^7$, $\ell = 3$




the carries do the balancing: $-\frac{38}{2^7} - \frac{51}{2^{14}} - \frac{26}{2^{21}} \equiv 0.6999998$

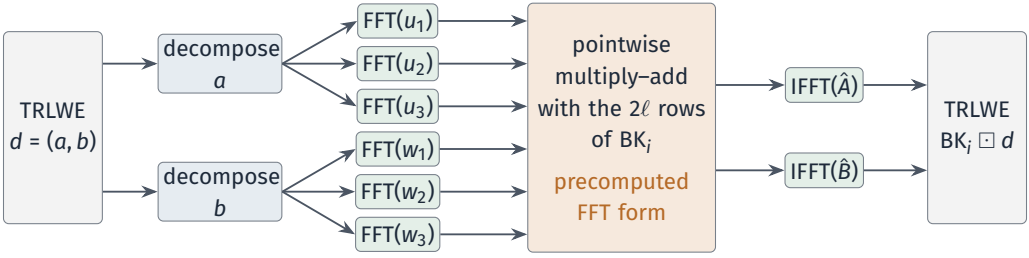
# The offset trick (2/2): $t = 0.7$, $B_g = 2^7$, $\ell = 3$



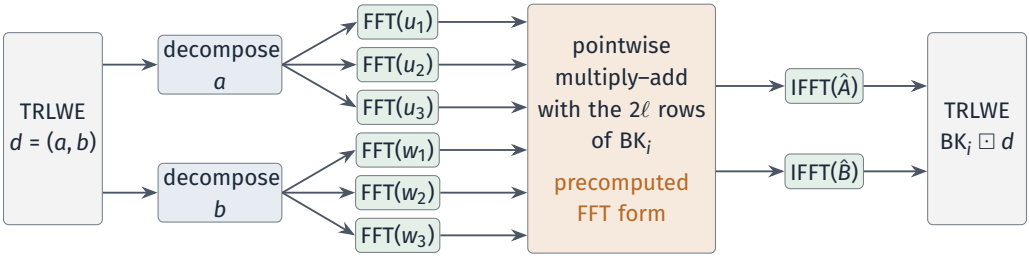

the carries do the balancing: $-\frac{38}{2^7} - \frac{51}{2^{14}} - \frac{26}{2^{21}} \equiv 0.6999998$

```
uint32_t t = p[j] + g.offset;                                  // the only addition
for (int i = 0; i < g.ell; ++i)
    out[i][j] = int32_t((t >> (32-(i+1)*g.Bgbit)) & g.mask) - int32_t(g.halfBg);
```

# External product in the frequency domain



# External product in the frequency domain




8 FFTs per external product $\times$ 630 per gate $\approx$ 5000 FFTs of size 512 per gate.

Price: BK takes 59.1 MiB, twice its coefficient form.

# Evaluation

# Setup and correctness

## Setup

- Intel Core Ultra 7 256V, one thread pinned to a performance core.
- GCC 15.3.1, `-O3 -march=native`.
- Original library, two FFT back ends:
  **spqlios-fma** (AVX2/FMA assembly),
  **nayuki-portable** (plain C).
- 5/3 runs (ours/original), 500 gates each.

# Setup and correctness

## Setup

- Intel Core Ultra 7 256V, one thread pinned to a performance core.
- GCC 15.3.1, `-O3 -march=native`.
- Original library, two FFT back ends:
  **spqlios-fma** (AVX2/FMA assembly),
  **nayuki-portable** (plain C).
- 5/3 runs (ours/original), 500 gates each.

## Correctness

- All 10 unit tests pass; 50 chained NANDs; majority circuit.
- Adder: 173 + 94 = 267 (8 bits) in 0.72 s, 9999999 + 7777777 = 17777776 (24 bits) in 2.16 s.
- 0 decryption errors in 12,500 gates.

# Setup and correctness

## Setup

- Intel Core Ultra 7 256V, one thread pinned to a performance core.
- GCC 15.3.1, `-O3 -march=native`.
- Original library, two FFT back ends:
  **spqlios-fma** (AVX2/FMA assembly),
  **nayuki-portable** (plain C).
- 5/3 runs (ours/original), 500 gates each.

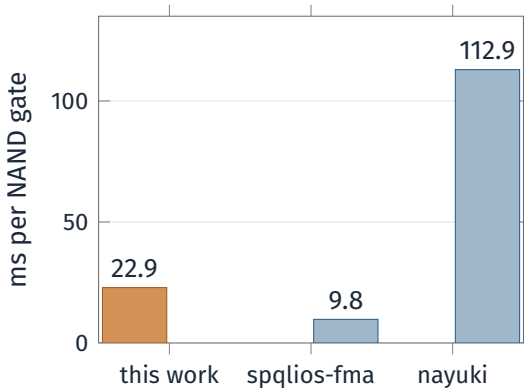
## Correctness

- All 10 unit tests pass; 50 chained NANDs; majority circuit.
- Adder: 173 + 94 = 267 (8 bits) in 0.72 s, 9999999 + 7777777 = 17777776 (24 bits) in 2.16 s.
- 0 decryption errors in 12,500 gates.

**Why no errors?**

Input std. dev. $\approx 0.005$ vs. margin $\frac{1}{8}$: **over 20 std. devs.**

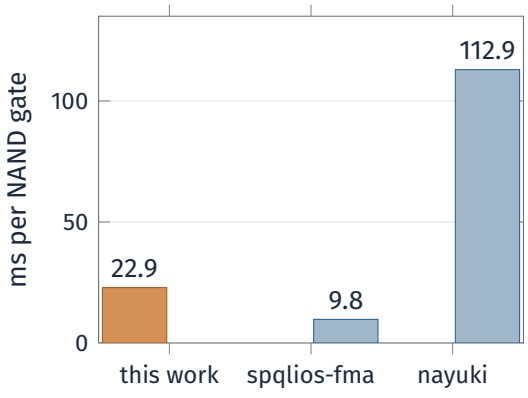
# Performance against the original library




- 2.3× slower than the assembly FFT.
- 4.9× faster than the portable FFT.

⇒ Same algorithm in all three: the **FFT decides**.

# Performance against the original library



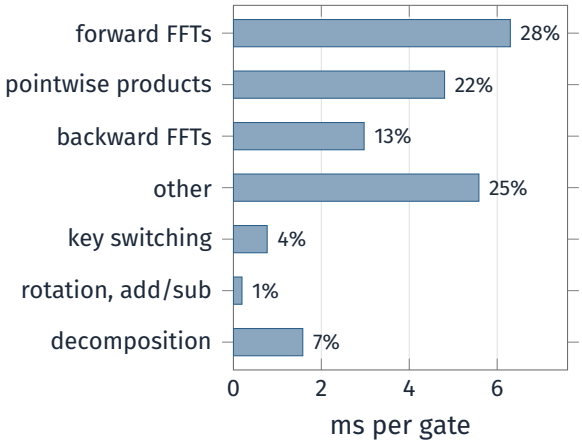

- 2.3× slower than the assembly FFT.
- 4.9× faster than the portable FFT.

⇒ Same algorithm in all three: the **FFT decides**.

| | ours | spqlios | nayuki |
|---|---|---|---|
| key gen. (s) | 0.21–0.22 | 0.53 | 0.88–0.94 |
| peak mem. (MiB) | 143 | 253 | 253 |

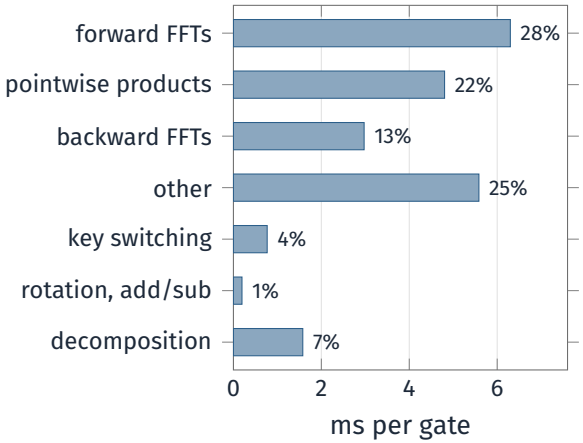
Keys: BK 59.1 MiB, KSK 78.9 MiB; one encrypted bit: 2524 bytes.

# Where the time goes




- Blind rotation: 96.5% of the gate.
- FFTs + pointwise products: 63%.
- Decomposition: only 7% (offset trick).

# Where the time goes




- Blind rotation: 96.5% of the gate.
- FFTs + pointwise products: 63%.
- Decomposition: only 7% (offset trick).

**Surprise: memory traffic**

Each step reads a different $BK_i$: the 59.1 MiB key is *streamed* every gate.

External product: 21.8 µs (key in cache) vs. 33.6 µs (as in blind rotation) $\approx$ 34% of the gate.

# Noise: bounds, predictions and measurements

- Worst-case bounds assume every digit has magnitude $\beta$.
- Digits are uniform: $\mathbb{E}[d^2] \approx \beta^2/3 \Rightarrow$ a third of the bound.

# Noise: bounds, predictions and measurements

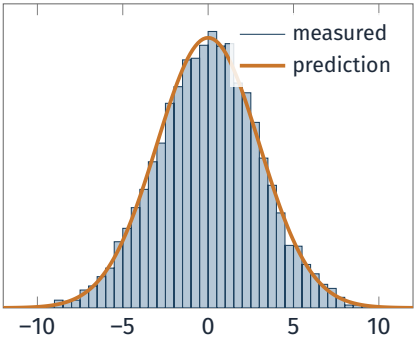
- Worst-case bounds assume every digit has magnitude $\beta$.
- Digits are uniform: $\mathbb{E}[d^2] \approx \beta^2/3 \Rightarrow$ a third of the bound.
- Key switching: fixed table noise, only $\frac{3}{4}$ of digits nonzero.

# Noise: bounds, predictions and measurements

- Worst-case bounds assume every digit has magnitude $\beta$.
- Digits are uniform: $\mathbb{E}[d^2] \approx \beta^2/3 \Rightarrow$ a third of the bound.
- Key switching: fixed table noise, only $\frac{3}{4}$ of digits nonzero.

| variance | bound | predicted | measured |
|---|---|---|---|
| blind rot. | $1.4 \cdot 10^{-5}$ | $4.7 \cdot 10^{-6}$ | $4.6 \cdot 10^{-6}$ |
| gate output | $2.2 \cdot 10^{-5}$ | $9.0 \cdot 10^{-6}$ | $9.0 \cdot 10^{-6}$ |

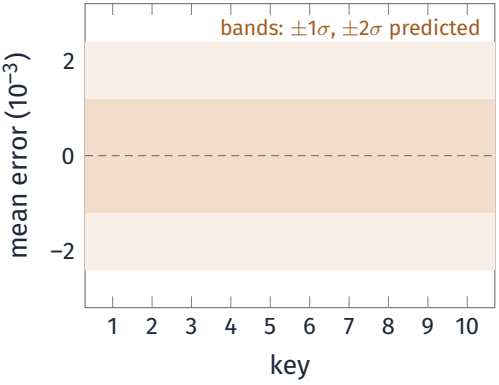
10 keys $\times$ 1,000 gates, within-key variance



# A key-dependent bias, and why the original has none

- Each KSK entry carries a fixed noise $e_{i,j,v}$.
- Averaging over inputs with the table fixed leaves a per-key bias:

$$\text{std} \approx \sqrt{3tN/16}\,\sigma_{\text{LWE}} = 1.2 \cdot 10^{-3}$$

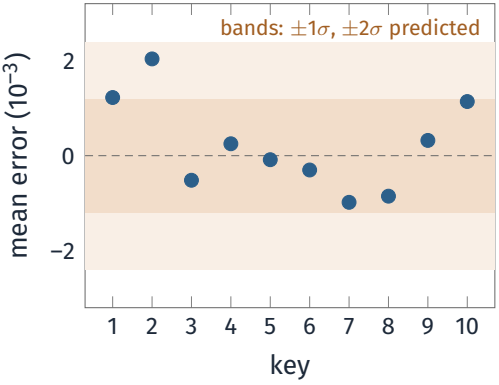


# A key-dependent bias, and why the original has none

- Each KSK entry carries a fixed noise $e_{i,j,v}$.
- Averaging over inputs with the table fixed leaves a per-key bias:

$$\text{std} \approx \sqrt{3tN/16}\,\sigma_{\text{LWE}} = 1.2 \cdot 10^{-3}$$

- Measured over 10 keys: $9.8 \cdot 10^{-4}$.

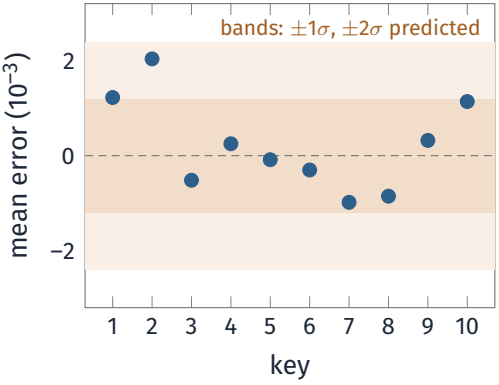


# A key-dependent bias, and why the original has none

- Each KSK entry carries a fixed noise $e_{i,j,v}$.
- Averaging over inputs with the table fixed leaves a per-key bias:

$$\text{std} \approx \sqrt{3tN/16}\,\sigma_{\text{LWE}} = 1.2 \cdot 10^{-3}$$

- Measured over 10 keys: $9.8 \cdot 10^{-4}$.
- The original library **centers** its key-switching noise $\Rightarrow$ mean error $\leq 2.0 \cdot 10^{-4}$ vs. ours up to $1.2 \cdot 10^{-3}$.
- Harmless: $100\times$ below the margin.



# Limitations and future work

**Limitations**

- `mt19937_64` is not a CSPRNG.
- No constant-time guarantees on the client.
- No serialization: client and server share a process.
- Parameters: $\approx$ 128 bits as estimated by the library authors; re-check with a current lattice estimator.

**Next steps** (by expected gain per effort)

1. Center the key-switching noise.
2. Remove allocations from the inner loops.
3. Contiguous BK + prefetch the next TRGSW sample.
4. Vectorize the FFT (AVX2/FMA) or radix-4.
5. Independent gates in parallel threads.
6. Programmable bootstrapping.

# Takeaways

**It works.**
TFHE from scratch in 526 lines; 22.9 ms per gate; 12,500 gates, 0 errors.

**Noise is predictable.**
Measured variances match an average-case analysis, which also explains a key-dependent bias.

**Small tricks matter.**
The offset trick makes balanced decomposition a single addition; the FFT-domain key saves most transforms.

**Performance lives in the FFT…**
…and in memory: 2.3× from assembly, 4.9× ahead of portable; the 59 MiB key is streamed every gate.

# Thank you!

## Questions?

Matheus de Oliveira Saldanha • MO422 • Unicamp

code, tests, report and these slides: `learning-tfhe`

# Backup: parameters

| Symbol | Value | Role |
|---|---|---|
| $n$ | 630 | LWE dimension; external products per bootstrapping |
| $N$ | 1024 | ring degree; the phase is rounded to $2N$ positions |
| $\ell$, $B_g$ | 3, $2^7$ | gadget decomposition of the external product |
| $t$, base | 8, $2^2$ | decomposition of the key switching |
| $\sigma_{\text{LWE}}$ | $2^{-15}$ | noise of LWE samples and of KSK |
| $\sigma_{\text{BK}}$ | $2^{-25}$ | noise of the TRGSW samples of BK |

The same as `new_default_gate_bootstrapping_parameters(128)` of the original library.

# Backup: noise of a bootstrapped gate

| | worst-case bound | average case |
|---|---|---|
| blind rotation | $n\left(2\ell N\beta^2\vartheta_{\mathrm{BK}} + (1+N)\varepsilon^2\right) = 1.4\cdot 10^{-5}$ | $n\,2\ell N\frac{\beta^2}{3}\vartheta_{\mathrm{BK}} = 4.7\cdot 10^{-6}$ |
| key switching | $tN\vartheta_{\mathrm{KSK}} + N\,2^{-2(t\cdot\mathrm{basebit}+1)}$ | $tN\left(\frac{B-1}{B}\right)^2\vartheta_{\mathrm{KSK}} = 4.3\cdot 10^{-6}$ |
| per-key bias | — | $\sqrt{tN\frac{B-1}{B^2}\vartheta_{\mathrm{KSK}}} = 1.2\cdot 10^{-3}$ (std.) |
| rounding to $\mathbb{Z}_{2N}$ | $\frac{1+n/2}{48N^2} \approx 6.3\cdot 10^{-6}$ (input side only) | |

NAND input: two gate outputs + rounding $\Rightarrow \sigma \approx 0.005 \ll \frac{1}{8}$.